



ओके सो आई थिंक पार्ट थ्री है ...
TXT 84.9KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Data Ingestion Module — Complete Notes

Project Part 3 — Building the first component of the pipeline: Data Ingestion

Big Picture: Where Are We?

In the previous video(s), we already completed:

1.  **MongoDB setup** (database + collection with data uploaded)
2.  **Logging module** (custom logger for tracking execution)
3.  **Exception module** (custom exception handling for debugging)
4.  **Experiment notebooks** (explored data in Jupyter)

Now we begin the **first real pipeline component: Data Ingestion**.

The Standard Workflow for Building Any Component

The instructor follows a repeatable workflow (introduced on day 1). For every component we build, the steps are:

text

1. Define constants (in constants/__init__.py)
2. Write configuration code (in config/ folder)
3. Add code to config entity (config_entity.py)
4. Add artifact entity (artifact_entity.py)
5. Build the component (components/ folder)
6. Build the pipeline (pipeline/ folder)
7. Run via demo.py

For **Data Ingestion**, the steps we complete in this video are:

Step	File/Folder	Purpose
1	src/constants/__init__.py	Store all constant values (DB name, collection name, file paths, split ratio, etc.)
2	src/configuration/mongo_db_connection.py	Establish connection to MongoDB
3	src/data_access/proj1_data.py	Fetch data from MongoDB as a Pandas DataFrame
4	src/entity/config_entity.py	Define DataIngestionConfig dataclass
5	src/entity/artifact_entity.py	Define DataIngestionArtifact dataclass
6	src/components/data_ingestion.py	The actual ingestion logic (export to feature store + train/test split)
7	src/pipeline/training_pipeline.py	Orchestrates components (only Data Ingestion for now)
8	demo.py	Entry point to run the pipeline

1 Constants — src/constants/__init__.py

Why a separate constants file?

- The same value (e.g., database name `Proj1`) may be used in **5+ places** across the project.
- Writing it everywhere = **code redundancy** and **maintenance pain**.
- Solution: define once, import everywhere.

Why __init__.py in every folder?

Because we added our **local packages** to the virtual environment. For Python to treat a folder as an importable package, it needs `__init__.py` . This is why you see it in:

- constants/
- components/
- configuration/
- data_access/
- entity/
- etc.

Note: Not every folder needs it, but almost all do in this project structure.

What goes inside constants?

Only **variable definitions and their values** — no logic. Example constants mentioned:

- DATABASE_NAME = "Proj1"
- COLLECTION_NAME = "Proj1-Data"
- MONGODB_URL (connection string — but sourced from environment variable)
- DATA_INGESTION_DIR_NAME
- DATA_INGESTION_FEATURE_STORE_DIR
- DATA_INGESTION_INGESTED_DIR
- TRAIN_TEST_SPLIT_RATIO
- TRAIN_FILE_NAME
- TEST_FILE_NAME
- ARTIFACT_DIR (root artifact folder)
- File names like data.csv

Key Insight: The MONGODB_URL is an Environment Variable

Instead of hardcoding the connection string (which is bad practice in production), we read it from an **environment variable**. This is a **production-grade technique**.

2 MongoDB Connection — src/configuration/mongo_db_connection.py

Purpose

Only one job: Establish a connection to MongoDB. Nothing else.

The MongoClient Class

python

```
class MongoClient:
    client = None # class-level variable (singleton pattern)

    def __init__(self, database_name=DATABASE_NAME) -> None:
        try:
            # If no client exists yet, create one
            if MongoClient.client is None:
                mongo_db_url = os.getenv("MONGODB_URL") # read from env var
                if mongo_db_url is None:
                    raise Exception("Environment variable MONGODB_URL is not set.")
                MongoClient.client = pymongo.MongoClient(mongo_db_url)
            self.client = MongoClient.client
            self.database = self.client[database_name]
            self.database_name = database_name
        except Exception as e:
            raise CustomException(e, sys)
```

Key Concepts

Concept	Explanation
Singleton pattern	<code>client = None</code> at class level. Only creates a new connection if none exists. Reuses the same connection across the project.
Environment variable	<code>os.getenv("MONGODB_URL")</code> — reads the connection string from the OS environment, not hardcoded.
Exception handling	Wraps everything in try/except and raises our custom <code>CustomException</code> with <code>sys</code> for traceback.

Concept	Explanation
Database name as default arg	database_name=DATABASE_NAME — defaults to the constant, but can be overridden.

Why environment variables?

- **Security:** Never commit passwords/connection strings to Git.
- **Flexibility:** Different environments (dev/staging/prod) can use different URLs without code changes.
- **Production standard:** Every real job uses this.

How to set the environment variable

PowerShell (Windows):

```
powershell
```

```
$env:MONGODB_URL = "mongodb+srv://<username>:<password>@cluster..."
```

Bash (Linux/Mac):

```
bash
```

```
export MONGODB_URL="mongodb+srv://<username>:<password>@cluster..."
```

Verify it's set:

```
powershell
```

```
echo $env:MONGODB_URL # PowerShell
```

```
echo $MONGODB_URL # Bash
```

Important: Set it inside the active virtual environment, not the base environment. Otherwise the venv won't see it.

Manual Windows method: Settings → Environment Variables → New → add MONGODB_URL with the full connection string.

⚠ **Common mistake:** Replacing only the username/password but not the full string. You must use the **entire connection string** (including `mongodb+srv://` , cluster host, etc.) and only swap `<username>` and `<password>` .

3 Data Access — `src/data_access/proj1_data.py`

Purpose

Use the MongoDB connection to **fetch data** and return it as a **Pandas DataFrame**.

The Proj1Data Class

python

```
class Proj1Data:
    def __init__(self):
        try:
            self.mongo_client = MongoClient(database_name=DATABASE_NAME)
        except Exception as e:
            raise CustomException(e, sys)

    def export_collection_as_dataframe(self, collection_name, database_name=None) -> pd.DataFrame:
        try:
            if database_name is None:
                collection = self.mongo_client.database[collection_name]
            else:
                collection = self.mongo_client.client[database_name][collection_name]

            df = pd.DataFrame(list(collection.find()))

            # Drop MongoDB's internal _id column
            if "_id" in df.columns:
                df = df.drop(columns=["_id"], axis=1)

            # Replace "na" strings with NaN
            df.replace({"na": np.nan}, inplace=True)

            return df
        except Exception as e:
            raise CustomException(e, sys)
```

Key Concepts

Concept	Explanation
<code>collection.find()</code>	MongoDB method that returns a cursor of all documents.
<code>list(...)</code>	Converts the cursor into a list of dicts.
<code>pd.DataFrame(...)</code>	Converts list of dicts into a DataFrame.
Drop <code>_id</code>	MongoDB auto-generates <code>_id</code> ; we don't need it for ML.
Replace <code>"na"</code>	Some datasets store missing values as the string <code>"na"</code> ; we convert to proper <code>NaN</code> .
Reuses <code>MongoDBClient</code>	This file doesn't create its own connection — it uses the one from <code>mongo_db_connection.py</code> .

Flow

text

```

Proj1Data.__init__()
    └─ MongoDBClient(database_name="Proj1")
        └─ creates/reuses pymongo client
            └─ self.database = client["Proj1"]

export_collection_as_dataframe("Proj1-Data")
    └─ collection = self.mongo_client.database["Proj1-Data"]
        └─ df = pd.DataFrame(list(collection.find()))
            └─ return df

```

Config Entity — `src/entity/config_entity.py`

Purpose

Define **dataclasses** that hold configuration values for each component. These dataclasses are instantiated in the component's `__init__` and carry all

paths/parameters.

TrainingPipelineConfig

python

```
@dataclass
class TrainingPipelineConfig:
    pipeline_name: str = PIPELINE_NAME
    artifact_dir: str = os.path.join(ARTIFACT_DIR, TIMESTAMP)
    timestamp: str = TIMESTAMP
```

- pipeline_name — name of the pipeline.
- artifact_dir — root artifact directory **with a timestamp** (e.g., artifact/12-29-2024_12-30-45/).
- timestamp — captured at runtime.

Why timestamp? It gives us a **versioning workaround**. Every run creates a new timestamped folder, so we can compare runs. (Tools like DVC are better, but this is a hardcoded alternative.)

DataIngestionConfig

python

```
@dataclass
class DataIngestionConfig:
    data_ingestion_dir: str = os.path.join(
        training_pipeline_config.artifact_dir, DATA_INGESTION_DIR_NAME
    )
    feature_store_file_path: str = os.path.join(
        data_ingestion_dir, DATA_INGESTION_FEATURE_STORE_DIR, FILE_NAME
    )
    training_file_path: str = os.path.join(
        data_ingestion_dir, DATA_INGESTION_INGESTED_DIR, TRAIN_FILE_NAME
    )
    testing_file_path: str = os.path.join(
        data_ingestion_dir, DATA_INGESTION_INGESTED_DIR, TEST_FILE_NAME
    )
    train_test_split_ratio: float = DATA_INGESTION_TRAIN_TEST_SPLIT_RATIO
    collection_name: str = DATA_INGESTION_COLLECTION_NAME
    database_name: str = DATABASE_NAME
```

This defines **all paths and parameters** the Data Ingestion component needs:

- Where to save the raw feature store file
- Where to save train/test files
- The split ratio
- Which collection/database to read from

5 Artifact Entity — src/entity/artifact_entity.py

Purpose

Define what each component **returns** (its outputs/artifacts).

```
python
```

```
@dataclass
class DataIngestionArtifact:
    trained_file_path: str
    test_file_path: str
```

- Data Ingestion returns two paths: the training file and the testing file.
- These paths are then passed to the **next component** (Data Validation) in future videos.

Why dataclasses for artifacts? They're clean, typed, and self-documenting. The pipeline knows exactly what each component produces.

6 The Component — src/components/data_ingestion.py

Purpose

The actual **Data Ingestion logic**. Two main methods:

1. `export_data_into_feature_store()` — fetch from MongoDB and save raw data.
2. `split_data_as_train_test()` — split into train/test and save separately.
3. `initiate_data_ingestion()` — orchestrates both and returns the artifact.

Code

python

```
class DataIngestion:
    def __init__(self, data_ingestion_config: DataIngestionConfig):
        try:
            self.data_ingestion_config = data_ingestion_config
        except Exception as e:
            raise CustomException(e, sys)

    def export_data_into_feature_store(self) -> pd.DataFrame:
        try:
            logging.info("Exporting data from MongoDB")
            my_data = Proj1Data()
            dataframe = my_data.export_collection_as_dataframe(
                collection_name=self.data_ingestion_config.collection_name
            )
            logging.info(f"Shape of dataframe: {dataframe.shape}")
            feature_store_file_path = self.data_ingestion_config.feature_store_file_path
            dir_path = os.path.dirname(feature_store_file_path)
            os.makedirs(dir_path, exist_ok=True)
            dataframe.to_csv(feature_store_file_path, index=False, header=True)
            return dataframe
        except Exception as e:
            raise CustomException(e, sys)

    def split_data_as_train_test(self, dataframe: pd.DataFrame) -> None:
        try:
            train_set, test_set = train_test_split(
                dataframe,
                test_size=self.data_ingestion_config.train_test_split_ratio
            )
            logging.info("Performed train test split")
            dir_path = os.path.dirname(self.data_ingestion_config.training_file_path)
            os.makedirs(dir_path, exist_ok=True)
            train_set.to_csv(
                self.data_ingestion_config.training_file_path,
                index=False, header=True
            )
            test_set.to_csv(
                self.data_ingestion_config.testing_file_path,
                index=False, header=True
            )
            logging.info("Exported train and test file path")
        except Exception as e:
            raise CustomException(e, sys)
```

```
def initiate_data_ingestion(self) -> DataIngestionArtifact:
    try:
        dataframe = self.export_data_into_feature_store()
        self.split_data_as_train_test(dataframe=dataframe)
        data_ingestion_artifact = DataIngestionArtifact(
            trained_file_path=self.data_ingestion_config.training_file_path,
            test_file_path=self.data_ingestion_config.testing_file_path
        )
        logging.info(f"Data ingestion artifact: {data_ingestion_artifact}")
        return data_ingestion_artifact
    except Exception as e:
        raise CustomException(e, sys)
```

What each method does

Method	What it does
<code>__init__</code>	Stores the <code>DataIngestionConfig</code> instance.
<code>export_data_into_feature_store</code>	Fetches data via <code>Proj1Data</code> , logs shape, creates the feature-store directory, saves CSV, returns <code>DataFrame</code> .
<code>split_data_as_train_test</code>	Uses <code>train_test_split</code> from <code>sklearn</code> , creates directories, saves train/test CSVs.
<code>initiate_data_ingestion</code>	Calls the two methods above and returns a <code>DataIngestionArtifact</code> with the two paths.

Key Concepts

- `os.makedirs(dir_path, exist_ok=True)` — creates nested directories safely (won't error if they exist).
- `train_test_split` — from `sklearn.model_selection`; uses the ratio from config.
- **Artifact returned** — the pipeline uses this to pass paths to the next component.
- **Logging at every step** — so we can trace execution.

7 The Pipeline — `src/pipeline/training_pipeline.py`

Purpose

Orchestrate components in order. For now, only Data Ingestion exists.

python

```
class TrainingPipeline:
    def __init__(self):
        self.training_pipeline_config = TrainingPipelineConfig()

    def start_data_ingestion(self) -> DataIngestionArtifact:
        try:
            self.data_ingestion_config = DataIngestionConfig(
                training_pipeline_config=self.training_pipeline_config
            )
            logging.info("Starting data ingestion")
            data_ingestion = DataIngestion(
                data_ingestion_config=self.data_ingestion_config
            )
            data_ingestion_artifact = data_ingestion.initiate_data_ingestion()
            logging.info(f"Data ingestion completed and artifact: {data_ingestion_artifact}")

            return data_ingestion_artifact
        except Exception as e:
            raise CustomException(e, sys)

    def run_pipeline(self):
        try:
            data_ingestion_artifact = self.start_data_ingestion()
            # Future components (commented out for now):
            # data_validation_artifact = self.start_data_validation(...)
            # data_transformation_artifact = self.start_data_transformation(...)
            # model_trainer_artifact = self.start_model_trainer(...)
        except Exception as e:
            raise CustomException(e, sys)
```

Key Concepts

- `TrainingPipelineConfig` — created once, holds the timestamped artifact dir.
- `start_data_ingestion` — creates `DataIngestionConfig`, instantiates `DataIngestion`, runs it, returns artifact.
- `run_pipeline` — the single entry method. For now only calls data ingestion. Future components are commented out.

- **Separation of concerns:** pipeline orchestrates, component does the work.

8 Entry Point — demo.py

python

```
from src.pipeline.training_pipeline import TrainingPipeline
```

```
if __name__ == "__main__":  
    pipeline = TrainingPipeline()  
    pipeline.run_pipeline()
```

This is what you run:

bash

```
python demo.py
```

The Debugging Journey (Very Important Lesson)

The Error

When first run, the pipeline crashed with:

text

```
pymongo.errors.ConfigurationError: The DNS query name does not exist
```

The traceback walked through every file:

text

```
demo.py line 28  
  → training_pipeline.py line 72  
    → training_pipeline.py line 59  
      → training_pipeline.py line 47  
        → data_ingestion.py line 86  
          → data_ingestion.py line 34  
            → proj1_data.py line 20
```

→ mongo_db_connection.py line 54
→ DNS query name does not exist

Root Cause

We used `os.getenv("MONGODB_URL")` but **never set the environment variable** `MONGODB_URL` .

The Fix

Set the environment variable in the active virtual environment:

```
powershell
```

```
$env:MONGODB_URL = "mongodb+srv://<username>:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority"
```

Why This Is a Great Lesson

- Our **custom exception + logging modules** made the error traceable file-by-file.
- This is exactly how real engineers debug: read the traceback, follow the chain, find the root cause.
- The instructor intentionally left the error in the recording to demonstrate debugging.

The Second Error (Typo)

After setting the env var, a new error appeared:

```
text
```

```
Password must be escaped according to RFC 3986
```

Cause: The instructor accidentally replaced only the username/password but not the full connection string.

Fix: Use the **entire connection string** (including `mongodb+srv://` , cluster host, options) and only swap `<username>` and `<password>` .

Final Folder Structure

text

```

project/
├─ artifact/                                # auto-created at runtime
│   └─ 12-29-2024_12-29-45/                # timestamped run
│       └─ data_ingestion/
│           ├─ feature_store/
│           │   └─ data.csv                 # raw data from MongoDB
│           └─ ingested/
│               ├─ train.csv                # training split
│               └─ test.csv                # testing split
├─ logs/                                    # auto-created
│   └─ 12_29_2024_12_29_45.log
├─ src/
│   ├─ constants/__init__.py
│   ├─ configuration/
│   │   └─ mongo_db_connection.py
│   ├─ data_access/
│   │   └─ proj1_data.py
│   ├─ entity/
│   │   ├─ config_entity.py
│   │   └─ artifact_entity.py
│   ├─ components/
│   │   └─ data_ingestion.py
│   ├─ pipeline/
│   │   └─ training_pipeline.py
│   ├─ exception/__init__.py
│   ├─ logging/__init__.py
│   └─ utils/__init__.py
├─ demo.py
├─ .gitignore                              # must include artifact/
└─ requirements.txt

```

Key Takeaways

1. **Constants file** = single source of truth. No hardcoding.
2. **Environment variables** = production-grade way to store secrets (MongoDB URL).
3. **Config entity** = dataclass holding all paths/params for a component.
4. **Artifact entity** = dataclass holding what a component returns.
5. **Component** = the actual logic. Pipeline = orchestration.
6. **Timestamped artifact dirs** = poor-man's data versioning (DVC is better).

7. **Custom exception + logging** = makes debugging across many files manageable.
8. **Singleton MongoDB client** = efficient connection reuse.
9. **.gitignore** = must ignore artifact/ , logs/ , .env , etc.
10. **Run via** `demo.py` = clean entry point.

✅ What's Done vs What's Next

Done	Next (future videos)
Constants	Data Validation component
MongoDB connection	Data Transformation component
Data access layer	Model Trainer component
Config + Artifact entities	Model Evaluation
Data Ingestion component	Model Pusher / Deployment
Training Pipeline (partial)	Prediction Pipeline
demo.py running end-to-end	

🔧 Homework (from the video)

- Replicate the entire Data Ingestion flow on your end.
- Push your code to the repo.
- Complete it within **2 days** so we can move to the next component.
- Any doubts → comment below the video.

💡 Bonus: Production Reality Check

"In my job, we use SSMS (SQL Server Management Studio) and define connection strings/user credentials as environment variables, connect, and pull data. For this

project, I chose MongoDB because many people need to learn it."

This reinforces: **the pattern is the same regardless of database** — environment variables + config + connection layer + data access layer + component + pipeline.

End of notes for Part 3 — Data Ingestion.



ओके चलो स्टार्ट करते हैं सो अ...
TXT 127.48KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Part 4 — Data Validation, Data Transformation & Model Trainer

Project: Full-fledged Production-Grade ML Project (Part 3 of the series)

Prerequisite: Part 1 (Introduction), Part 2 (MongoDB Setup + Data Ingestion)

Big Picture — Where We Are Now

In the previous video, we completed **Data Ingestion**. Now, following the same workflow, we build **three more components** in this video:

text

Data Ingestion  → Data Validation → Data Transformation → Model Trainer

After Model Trainer, the next steps (in future videos) will be:

text

Model Evaluation → Model Pusher (AWS) → Deployment

The Repeatable Workflow (used for every component)

text

1. Constants (src/constants/__init__.py)
2. Config code (src/configuration/...)
3. Config Entity (src/entity/config_entity.py)
4. Artifact Entity (src/entity/artifact_entity.py)
5. Component (src/components/...)
6. Pipeline (src/pipeline/training_pipeline.py)
7. Run via demo.py

Step 0 — Setup Before Building Components

Before jumping into Data Validation, two setup files must be prepared:

A. src/utils/main_utils.py

A utility file containing **common reusable functions** used across multiple components. Key methods include:

Method	Purpose
<code>read_yaml_file(file_path)</code>	Opens a YAML file and returns its contents as a dict
<code>write_yaml_file(file_path, content)</code>	Writes content to a YAML file
<code>save_object(file_path, obj)</code>	Saves a Python object (e.g., scaler) using pickle
<code>load_object(file_path)</code>	Loads a pickled object
<code>save_numpy_array_data(file_path, array)</code>	Saves a NumPy array as .npy
<code>load_numpy_array_data(file_path)</code>	Loads a .npy file

Why keep these? These are **plug-and-play** — you can reuse them in any future project. Not all methods are used in this project, but having them as a stock is useful.

Key usage in this project: `read_yaml_file` is used to read `schema.yaml`.

B. config/schema.yaml

A YAML file defining the **expected schema** of the data. Contains:

```
yaml
```

```
columns:
```

- Gender
- Age
- Driving_License
- ... (all expected columns)

```
numerical_columns:
```

- Age
- Annual_Premium
- Vintage
- ...

```
categorical_columns:
```

- Gender
- Vehicle_Age
- Vehicle_Damage
- ...

```
drop_columns:
```

- _id

```
num_features:
```

- Age
- Annual_Premium
- Vintage

```
mm_columns:
```

- Annual_Premium

Why this file? It's used by:

- **Data Validation** → to check expected columns/data types exist.
- **Data Transformation** → to know which columns to scale (StandardScaler vs MinMaxScaler), which to drop, which are categorical.

1 Data Validation Component

Purpose

Ensure the data pulled from MongoDB has the **expected columns and data types**. If not, fail early and produce a report.

Workflow

text

Constants → Config Entity → Artifact Entity → Component → Pipeline → Run

A. Constants (already defined in Part 3)

Data validation related constants already exist:

- DATA_VALIDATION_DIR_NAME
- DATA_VALIDATION_REPORT_FILE_NAME (e.g., report.yaml)

B. Config Entity — DataValidationConfig

python

```
@dataclass
class DataValidationConfig:
    data_validation_dir: str = os.path.join(
        training_pipeline_config.artifact_dir, DATA_VALIDATION_DIR_NAME
    )
    validation_report_file_path: str = os.path.join(
        data_validation_dir, DATA_VALIDATION_REPORT_FILE_NAME
    )
```

- data_validation_dir → artifact/<timestamp>/data_validation/
- validation_report_file_path → .../data_validation/report.yaml

C. Artifact Entity — DataValidationArtifact

python

```
@dataclass
class DataValidationArtifact:
    validation_status: bool
```

```
message: str
validation_report_file_path: str
```

- validation_status → True / False
- message → reason if failed
- validation_report_file_path → where the YAML report is saved

D. The Component — src/components/data_validation.py

python

```
class DataValidation:
    def __init__(self, data_validation_config: DataValidationConfig,
                  data_ingestion_artifact: DataIngestionArtifact):
        try:
            self.data_validation_config = data_validation_config
            self.data_ingestion_artifact = data_ingestion_artifact
            self.schema_config = read_yaml_file(SCHEMA_FILE_PATH)
        except Exception as e:
            raise CustomException(e, sys)

    @staticmethod
    def read_data(file_path) -> pd.DataFrame:
        try:
            return pd.read_csv(file_path)
        except Exception as e:
            raise CustomException(e, sys)

    def validate_number_of_columns(self, dataframe: pd.DataFrame) -> bool:
        try:
            number_of_columns = len(self.schema_config["columns"])
            logging.info(f"Required number of columns: {number_of_columns}")
            logging.info(f>Data frame has columns: {len(dataframe.columns)}")
            if len(dataframe.columns) == number_of_columns:
                return True
            return False
        except Exception as e:
            raise CustomException(e, sys)

    def is_column_exist(self, df: pd.DataFrame) -> bool:
        try:
            dataframe_columns = df.columns
            missing_numerical_columns = []
            missing_categorical_columns = []
```

```

for column in self.schema_config["numerical_columns"]:
    if column not in dataframe_columns:
        missing_numerical_columns.append(column)

if len(missing_numerical_columns) > 0:
    logging.info(f"Missing numerical column: {missing_numerical_columns}")

for column in self.schema_config["categorical_columns"]:
    if column not in dataframe_columns:
        missing_categorical_columns.append(column)

if len(missing_categorical_columns) > 0:
    logging.info(f"Missing categorical column: {missing_categorical_columns}")

    return False if len(missing_numerical_columns) > 0 or len(missing_categorical_columns) > 0 else True
except Exception as e:
    raise CustomException(e, sys)

def initiate_data_validation(self) -> DataValidationArtifact:
    try:
        validation_error_msg = ""
        logging.info("Starting data validation")

        train_df = DataValidation.read_data(
            file_path=self.data_ingestion_artifact.trained_file_path)
        test_df = DataValidation.read_data(
            file_path=self.data_ingestion_artifact.test_file_path)

        # Check number of columns
        status = self.validate_number_of_columns(dataframe=train_df)
        if not status:
            validation_error_msg += f"Columns are missing in training dataframe. "
        else:
            logging.info(f"All required columns present in training dataframe: {status}")

        status = self.validate_number_of_columns(dataframe=test_df)
        if not status:
            validation_error_msg += f"Columns are missing in test dataframe. "
        else:
            logging.info(f"All required columns present in test dataframe: {status}")

        # Check column existence
        status = self.is_column_exist(df=train_df)
        if not status:

```

```

        validation_error_msg += f"Columns are missing in training dataframe. "
    else:
        logging.info(f"All categorical/int columns present in training dataframe: {status}")

    status = self.is_column_exist(df=test_df)
    if not status:
        validation_error_msg += f"Columns are missing in test dataframe. "
    else:
        logging.info(f"All categorical/int columns present in test dataframe: {status}")

    # Determine validation status
    validation_status = len(validation_error_msg) == 0

    data_validation_artifact = DataValidationArtifact(
        validation_status=validation_status,
        message=validation_error_msg,
        validation_report_file_path=self.data_validation_config.validation_report_file_path
    )

    # Save report
    report = {
        "validation_status": validation_status,
        "message": validation_error_msg.strip()
    }
    os.makedirs(os.path.dirname(self.data_validation_config.validation_report_file_path), exist_ok=True)
    write_yaml_file(
        file_path=self.data_validation_config.validation_report_file_path,
        content=report
    )

    logging.info("Data validation artifact created and saved to yaml file.")
    return data_validation_artifact
except Exception as e:
    raise CustomException(e, sys)

```

Key Concepts

Concept	Explanation
@staticmethod	read_data doesn't need self ; can be called without creating an instance.

Concept	Explanation
<code>read_data</code>	Reads CSV → DataFrame. Used for both train and test.
<code>validate_number_of_columns</code>	Compares schema's expected column count to dataframe's actual count.
<code>is_column_exist</code>	Checks each numerical and categorical column from schema exists in the dataframe.
<code>validation_error_msg</code>	Empty string if all good; accumulates error messages otherwise.
<code>validation_status = len(msg) == 0</code>	True only if no errors accumulated.
Report YAML	Saved to artifact/<timestamp>/data_validation/report.yaml with status + message.

E. Pipeline Integration — TrainingPipeline

python

```
def start_data_validation(self, data_ingestion_artifact: DataIngestionArtifact) -> DataValidationArtifact:
    try:
        self.data_validation_config = DataValidationConfig(
            training_pipeline_config=self.training_pipeline_config
        )
        data_validation = DataValidation(
            data_validation_config=self.data_validation_config,
            data_ingestion_artifact=data_ingestion_artifact
        )
        data_validation_artifact = data_validation.initiate_data_validation()
        logging.info(f"Data validation completed and artifact: {data_validation_artifact}")

    except Exception as e:
        raise CustomException(e, sys)
```

F. Run via demo.py

python

```
if __name__ == "__main__":  
    pipeline = TrainingPipeline()  
    pipeline.run_pipeline()
```

Expected Output

- artifact/<timestamp>/data_validation/report.yaml created
- Content:

yaml

```
validation_status: true  
message: ''
```

Try It Yourself (Test Failure)

Open config/schema.yaml and:

- Remove a column from columns list, or
- Rename Response to MyResponse , Or
- Move a column from categorical_columns to numerical_columns

Re-run → validation_status: false with a reasonable message.

2 Data Transformation Component

Purpose

Apply feature engineering from the experiment notebook:

- Map Gender to 0/1
- Drop unnecessary columns (_id)
- Create dummy variables for categorical columns

- Rename columns
- Apply StandardScaler to `num_features`
- Apply MinMaxScaler to `mm_columns`
- Handle class imbalance with **SMOTEENN**
- Save transformed train/test arrays and the preprocessing object

Why check Data Validation status first?

Before transformation, we **assert** `data_validation_artifact.validation_status == True`. If validation failed, there's no point in transforming bad data.

A. Constants

python

```
DATA_TRANSFORMATION_DIR_NAME = "data_transformation"
DATA_TRANSFORMATION_TRANSFORMED_DATA_DIR = "transformed"
DATA_TRANSFORMATION_TRANSFORMED_OBJECT_DIR = "transformed_object"
```

B. Config Entity — DataTransformationConfig

python

```
@dataclass
class DataTransformationConfig:
    data_transformation_dir: str = os.path.join(
        training_pipeline_config.artifact_dir, DATA_TRANSFORMATION_DIR_NAME
    )
    transformed_train_file_path: str = os.path.join(
        data_transformation_dir, DATA_TRANSFORMATION_TRANSFORMED_DATA_DIR, TRAIN_FILE_NAME
        .replace("csv", "npz")
    )
    transformed_test_file_path: str = os.path.join(
        data_transformation_dir, DATA_TRANSFORMATION_TRANSFORMED_DATA_DIR, TEST_FILE_NAME
        .replace("csv", "npz")
    )
    transformed_object_file_path: str = os.path.join(
        data_transformation_dir, DATA_TRANSFORMATION_TRANSFORMED_OBJECT_DIR, PREPROCESSING_OBJECT_FILE_NAME
    )
```

- Train/test saved as `.npy` (NumPy arrays) — efficient for ML.
- Preprocessing object saved as `.pkl` (pickle).

C. Artifact Entity — `DataTransformationArtifact`

python

```
@dataclass
class DataTransformationArtifact:
    transformed_object_file_path: str
    transformed_train_file_path: str
    transformed_test_file_path: str
```

D. The Component — `src/components/data_transformation.py`

python

```
class DataTransformation:
    def __init__(self,
                 data_transformation_config: DataTransformationConfig,
                 data_ingestion_artifact: DataIngestionArtifact,
                 data_validation_artifact: DataValidationArtifact):
        try:
            self.data_transformation_config = data_transformation_config
            self.data_ingestion_artifact = data_ingestion_artifact
            self.data_validation_artifact = data_validation_artifact
            self.schema_config = read_yaml_file(SCHEMA_FILE_PATH)
        except Exception as e:
            raise CustomException(e, sys)

    @staticmethod
    def read_data(file_path) -> pd.DataFrame:
        try:
            return pd.read_csv(file_path)
        except Exception as e:
            raise CustomException(e, sys)

    @staticmethod
    def get_data_transformer_object() -> Pipeline:
        try:
            numerical_columns = ["writing_score", "reading_score"]
            categorical_columns = [...] # from schema

            num_pipeline = Pipeline(steps=[
```

```

        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ])
    cat_pipeline = Pipeline(steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("one_hot_encoder", OneHotEncoder()),
        ("scaler", StandardScaler(with_mean=False))
    ])

    preprocessor = ColumnTransformer([
        ("num_pipeline", num_pipeline, numerical_columns),
        ("cat_pipeline", cat_pipeline, categorical_columns)
    ])
    return preprocessor
except Exception as e:
    raise CustomException(e, sys)

def initiate_data_transformation(self) -> DataTransformationArtifact:
    try:
        if not self.data_validation_artifact.validation_status:
            raise Exception(self.data_validation_artifact.message)

        train_df = DataTransformation.read_data(
            self.data_ingestion_artifact.trained_file_path)
        test_df = DataTransformation.read_data(
            self.data_ingestion_artifact.test_file_path)

        input_feature_train_df = train_df.drop(columns=[TARGET_COLUMN], axis=1)
        target_feature_train_df = train_df[TARGET_COLUMN]

        input_feature_test_df = test_df.drop(columns=[TARGET_COLUMN], axis=1)
        target_feature_test_df = test_df[TARGET_COLUMN]

        preprocessor = self.get_data_transformer_object()

        # Combine train and test temporarily to fit the preprocessor
        input_feature_train_arr = preprocessor.fit_transform(input_feature_train_df)
        input_feature_test_arr = preprocessor.transform(input_feature_test_df)

        # SMOTEENN for handling imbalance
        smt = SMOTEENN(sampling_strategy="minority")
        input_feature_train_final, target_feature_train_final = smt.fit_resample(
            input_feature_train_arr, target_feature_train_df)
        input_feature_test_final, target_feature_test_final = smt.fit_resample(
            input_feature_test_arr, target_feature_test_df)

        # Concatenate features + target

```

```
train_arr = np.c_[input_feature_train_final, np.array(target_feature_train_fi
nal)]

test_arr = np.c_[input_feature_test_final, np.array(target_feature_test_fina
l)]

# Save object
save_object(
    file_path=self.data_transformation_config.transformed_object_file_path,
    obj=preprocessor
)

# Save arrays
save_numpy_array_data(
    file_path=self.data_transformation_config.transformed_train_file_path,
    array=train_arr
)
save_numpy_array_data(
    file_path=self.data_transformation_config.transformed_test_file_path,
    array=test_arr
)

return DataTransformationArtifact(
    transformed_object_file_path=self.data_transformation_config.transformed_
object_file_path,
    transformed_train_file_path=self.data_transformation_config.transformed_t
rain_file_path,
    transformed_test_file_path=self.data_transformation_config.transformed_te
st_file_path
)
except Exception as e:
    raise CustomException(e, sys)
```

Key Concepts

Concept	Explanation
ColumnTransformer	Applies different transformers to different column subsets.
Pipeline	Chains multiple steps (imputer → scaler → encoder) into one object.
StandardScaler	Standardizes features: $(x - \text{mean}) / \text{std}$.

Concept	Explanation
MinMaxScaler	Scales features to a fixed range (default 0–1).
SimpleImputer	Fills missing values (median for numerical, most_frequent for categorical).
OneHotEncoder	Converts categorical variables into dummy/binary columns.
SMOTEENN	Hybrid resampling: SMOTE (oversample minority) + ENN (clean noisy samples).
np.c_[]	Concatenates arrays column-wise (features + target).
.npy	NumPy's binary format — efficient for large arrays.
.pkl	Pickle — saves Python objects (e.g., the fitted preprocessor).

Why save the preprocessor object?

- In **production/batch inference**, new data must be transformed **exactly the same way** as training data.
- Loading the saved preprocessor and calling `.transform()` on new data ensures consistency.

Pipeline Integration

python

```
def start_data_transformation(self,
                               data_ingestion_artifact: DataIngestionArtifact,
                               data_validation_artifact: DataValidationArtifact) -> DataTransformationArtifact:
    try:
        self.data_transformation_config = DataTransformationConfig(
            training_pipeline_config=self.training_pipeline_config
        )
        data_transformation = DataTransformation(
            data_transformation_config=self.data_transformation_config,
            data_ingestion_artifact=data_ingestion_artifact,
```

```

        data_validation_artifact=data_validation_artifact
    )
    data_transformation_artifact = data_transformation.initiate_data_transformation()
    logging.info(f"Data transformation completed and artifact: {data_transformation_a
rtifact}")
    return data_transformation_artifact
except Exception as e:
    raise CustomException(e, sys)

```

Expected Output

text

```

artifact/<timestamp>/data_transformation/
├── transformed/
│   ├── train.npy
│   └── test.npy
└── transformed_object/
    └── preprocessing.pkl

```

3 Model Trainer Component

Purpose

Load transformed arrays, train a **RandomForestClassifier**, evaluate it, and save the model if it beats the expected accuracy threshold.

A. Constants

python

```

MODEL_TRAINER_DIR_NAME = "model_trainer"
MODEL_TRAINER_TRAINED_MODEL_DIR = "trained_model"
MODEL_TRAINER_TRAINED_MODEL_NAME = "model.pkl"
MODEL_TRAINER_EXPECTED_SCORE = 0.6
MODEL_TRAINER_OVER_FITTING_UNDER_FITTING_THRESHOLD = 0.05

# RandomForest parameters
MODEL_TRAINER_N_ESTIMATORS = 200
MODEL_TRAINER_MIN_SAMPLES_SPLIT = 7
MODEL_TRAINER_MIN_SAMPLES_LEAF = 6

```

```
MODEL_TRAINER_MAX_DEPTH = 10
MODEL_TRAINER_MIN_SAMPLES_SPLIT = 7
MODEL_TRAINER_RANDOM_STATE = 101
```

B. Config Entity — ModelTrainerConfig

python

```
@dataclass
class ModelTrainerConfig:
    model_trainer_dir: str = os.path.join(
        training_pipeline_config.artifact_dir, MODEL_TRAINER_DIR_NAME
    )
    trained_model_file_path: str = os.path.join(
        model_trainer_dir, MODEL_TRAINER_TRAINED_MODEL_NAME
    )
    expected_accuracy: float = MODEL_TRAINER_EXPECTED_SCORE
    model_config_file_path: str = MODEL_TRAINER_MODEL_CONFIG_FILE_PATH
    n_estimators = MODEL_TRAINER_N_ESTIMATORS
    min_samples_split = MODEL_TRAINER_MIN_SAMPLES_SPLIT
    min_samples_leaf = MODEL_TRAINER_MIN_SAMPLES_LEAF
    max_depth = MODEL_TRAINER_MAX_DEPTH
    random_state = MODEL_TRAINER_RANDOM_STATE
    over_fitting_under_fitting_threshold = MODEL_TRAINER_OVER_FITTING_UNDER_FITTING_THRES
HOLD
```

C. Artifact Entity

Two artifacts are defined:

python

```
@dataclass
class ClassificationMetricArtifact:
    f1_score: float
    precision_score: float
    recall_score: float

@dataclass
class ModelTrainerArtifact:
    trained_model_file_path: str
    metric_artifact: ClassificationMetricArtifact
```

D. Estimator Entity — src/entity/estimator.py

python

```
class MyModel:
    def __init__(self, preprocessing_object: Pipeline, trained_model_object: object):
        self.preprocessing_object = preprocessing_object
        self.trained_model_object = trained_model_object

    def predict(self, X: pd.DataFrame) -> np.ndarray:
        transformed_feature = self.preprocessing_object.transform(X)
        return self.trained_model_object.predict(transformed_feature)

    def __repr__(self):
        return f"{type(self.trained_model_object).__name__}()"

    def __str__(self):
        return f"{type(self.trained_model_object).__name__}()"

```

Why wrap the model + preprocessor together?

- In production, when you call `model.predict(new_data)`, the preprocessor must be applied first.
- This wrapper ensures **both happen together** — no mistakes.

E. The Component — src/components/model_trainer.py

python

```
class ModelTrainer:
    def __init__(self,
                 model_trainer_config: ModelTrainerConfig,
                 data_transformation_artifact: DataTransformationArtifact):
        try:
            self.model_trainer_config = model_trainer_config
            self.data_transformation_artifact = data_transformation_artifact
        except Exception as e:
            raise CustomException(e, sys)

    def get_model_object_and_report(self, train: np.array, test: np.array) -> Tuple[object, object]:
        try:
            logging.info("Training RandomForestClassifier with specified parameters")
            x_train, y_train, x_test, y_test = (
                train[:, :-1], train[:, -1],

```

```

        test[:, :-1], test[:, -1]
    )

    model = RandomForestClassifier(
        n_estimators=self.model_trainer_config.n_estimators,
        min_samples_split=self.model_trainer_config.min_samples_split,
        min_samples_leaf=self.model_trainer_config.min_samples_leaf,
        max_depth=self.model_trainer_config.max_depth,
        random_state=self.model_trainer_config.random_state
    )
    model.fit(x_train, y_train)
    y_pred = model.predict(x_test)

    accuracy = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred)
    recall = recall_score(y_test, y_pred)

    metric_artifact = ClassificationMetricArtifact(
        f1_score=f1, precision_score=precision, recall_score=recall
    )
    return model, metric_artifact
except Exception as e:
    raise CustomException(e, sys)

def initiate_model_trainer(self) -> ModelTrainerArtifact:
    try:
        logging.info("Entered initiate_model_trainer method of ModelTrainer class")
        logging.info("Unzipping data")

        train_arr = load_numpy_array_data(
            file_path=self.data_transformation_artifact.transformed_train_file_path)
        test_arr = load_numpy_array_data(
            file_path=self.data_transformation_artifact.transformed_test_file_path)

        trained_model, metric_artifact = self.get_model_object_and_report(
            train=train_arr, test=test_arr)

        preprocessing_obj = load_object(
            file_path=self.data_transformation_artifact.transformed_object_file_path)

        if accuracy_score(train_arr[:, -1], trained_model.predict(train_arr[:, :-1]))
< self.model_trainer_config.expected_accuracy:
            logging.info("No model found with score above the base score")
            raise Exception("No model found with score above the base score")

        # Wrap model + preprocessor

```

```
new_model = MyModel(
    preprocessing_object=preprocessing_obj,
    trained_model_object=trained_model
)

save_object(
    file_path=self.model_trainer_config.trained_model_file_path,
    obj=new_model
)

return ModelTrainerArtifact(
    trained_model_file_path=self.model_trainer_config.trained_model_file_pat
h,
    metric_artifact=metric_artifact
)
except Exception as e:
    raise CustomException(e, sys)
```

Key Concepts

Concept	Explanation
RandomForestClassifier	Ensemble of decision trees; robust and high-performing for tabular data.
accuracy_score , f1_score , precision_score , recall_score	Evaluation metrics from sklearn.
load_numpy_array_data	Loads .npy files back into NumPy arrays.
load_object	Loads pickled preprocessor.
MyModel wrapper	Ensures preprocessing always happens before prediction.
Expected accuracy check	If model's accuracy is below threshold, fail the pipeline.
Metric artifact	Stores f1, precision, recall for reporting.

Pipeline Integration

python

```

def start_model_trainer(self,
                        data_transformation_artifact: DataTransformationArtifact) -> ModelTrainerArtifact:
    try:
        self.model_trainer_config = ModelTrainerConfig(
            training_pipeline_config=self.training_pipeline_config
        )
        model_trainer = ModelTrainer(
            model_trainer_config=self.model_trainer_config,
            data_transformation_artifact=data_transformation_artifact
        )
        model_trainer_artifact = model_trainer.initiate_model_trainer()
        logging.info(f"Model trainer completed and artifact: {model_trainer_artifact}")
        return model_trainer_artifact
    except Exception as e:
        raise CustomException(e, sys)

```

Expected Output

text

```

artifact/<timestamp>/model_trainer/
├── trained_model/
│   └── model.pkl

```

Full Pipeline Flow

python

```

class TrainingPipeline:
    def run_pipeline(self):
        try:
            data_ingestion_artifact = self.start_data_ingestion()
            data_validation_artifact = self.start_data_validation(
                data_ingestion_artifact=data_ingestion_artifact
            )
            data_transformation_artifact = self.start_data_transformation(
                data_ingestion_artifact=data_ingestion_artifact,
                data_validation_artifact=data_validation_artifact
            )
            model_trainer_artifact = self.start_model_trainer(
                data_transformation_artifact=data_transformation_artifact
            )

```

```
except Exception as e:
    raise CustomException(e, sys)
```

Each component passes its **artifact** to the next — a clean chain of dependencies.

Final Artifact Structure

text

```
artifact/
├── <timestamp>/
│   ├── data_ingestion/
│   │   ├── feature_store/data.csv
│   │   └── ingested/{train.csv, test.csv}
│   ├── data_validation/
│   │   └── report.yaml
│   ├── data_transformation/
│   │   ├── transformed/{train.npy, test.npy}
│   │   └── transformed_object/preprocessing.pkl
│   └── model_trainer/
│       └── trained_model/model.pkl
```

Key Takeaways

1. **Same workflow every time** — constants → config → artifact → component → pipeline.
2. **Schema YAML** = single source of truth for column names/types/scaling needs.
3. **Data Validation** fails early if schema doesn't match — saves time.
4. **Data Transformation** = imputation + encoding + scaling + SMOTEENN + saving preprocessor.
5. **Preprocessor must be saved** for consistent production inference.
6. **SMOTEENN** handles class imbalance (80/20 → balanced).
7. `MyModel` **wrapper** bundles preprocessor + model for safe prediction.
8. **Model Trainer** checks accuracy threshold before saving.
9. **Artifacts chain** — each component's output feeds the next.
10. **Logging + CustomException** make debugging across many files manageable.

⚠ Common Mistakes to Avoid

Mistake	Fix
Forgetting <code>except</code> block in <code>try</code>	Always add <code>raise CustomException(e, sys)</code>
Hardcoding column names	Read from <code>schema.yaml</code>
Not saving preprocessor	Save it — needed for batch inference
Skipping validation check	Always assert <code>validation_status == True</code> before transformation
Using <code>.csv</code> for large arrays	Use <code>.npz</code> for NumPy arrays
Not wrapping model + preprocessor	Use <code>MyModel</code> wrapper for production

🧪 Homework

1. Replicate all three components on your end.
2. Test failure by modifying `schema.yaml` and re-running.
3. Push code to your repo.
4. Complete within **2 days** so we can start AWS in the next video.

🌟 What's Next

- **Model Evaluation** component
- **Model Pusher** component (push to AWS S3)
- **AWS setup** — services, repos
- **Deployment** — API, Docker, CI/CD

"From here on, we'll slowly move toward AWS, because the next steps are Model Evaluation and Model Pusher — pushing to AWS repos."

End of Part 4 Notes.



ओके सो वेलकम बैक टू दिस ...
TXT 97.45KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Part 5 — AWS Setup + Model Evaluation + Model Pusher

Project: Full-fledged Production-Grade ML Project (MLOps series)

Prerequisite: Parts 1–4 (Introduction → MongoDB → Data Ingestion → Validation → Transformation → Model Trainer)

Big Picture — Where We Are Now

Up to the previous video, we completed:

text

Data Ingestion  → Data Validation  → Data Transformation  → Model Trainer 

This video covers **three things**:

1. **AWS Setup** (IAM user, access keys, environment variables, S3 bucket)
2. **Model Evaluation** (compare new model vs. previously deployed model)
3. **Model Pusher** (push the accepted model to S3)

After this, the next steps are:

text

Prediction Pipeline → Application → CI/CD → Deployment

PART A — AWS Setup

Why AWS?

The next components need **remote storage** so that:

- Models are versioned and stored centrally.
- Previously deployed models can be compared against newly trained ones.
- Deployment can pull the latest accepted model.

Step 1 — Create an IAM User (Not Root!)

Concept: In any AWS account, you should **never** use the root user for daily work. Instead, create **IAM users** with restricted or admin policies.

Why?

- **Security:** If credentials leak, damage is limited.
- **Access control:** Different users get different permissions (dev, ops, data science).
- **Auditability:** You can track who did what.

Steps:

1. Log in to AWS Console with **root** (email).
2. **Select region:** `us-east-1` (**N. Virginia**) — critical because all resources must be in the same region.
3. Go to **IAM** → **Users** → **Create User**.
4. Name: `first-proj`.
5. **Do NOT** give Management Console access (we only need programmatic/CLI access).
6. Attach policy: `AdministratorAccess` (for personal projects; in real jobs, use least-privilege policies).
7. Create user.

Step 2 — Create Access Keys

Concept: Access keys are like a **username + password** for CLI/SDK access. They consist of:

- **Access Key ID** (public identifier)
- **Secret Access Key** (secret — never share)

Steps:

1. IAM → Users → first-proj → **Security credentials**.
2. **Create access key** → Choose **CLI** (we'll use terminal).
3. Download the `.csv` file — good practice in case you lose it.
4. Save the CSV somewhere safe (e.g., Desktop/ML-series).

Step 3 — Set Environment Variables

Like MongoDB URL earlier, AWS credentials go into **environment variables** (never hardcoded).

PowerShell:

```
powershell
```

```
$env:AWS_ACCESS_KEY_ID = "your_access_key_id"
$env:AWS_SECRET_ACCESS_KEY = "your_secret_access_key"
$env:AWS_DEFAULT_REGION = "us-east-1"
```

Bash:

```
bash
```

```
export AWS_ACCESS_KEY_ID="your_access_key_id"
export AWS_SECRET_ACCESS_KEY="your_secret_access_key"
export AWS_DEFAULT_REGION="us-east-1"
```

Verify:

```
powershell
```

```
echo $env:AWS_ACCESS_KEY_ID
echo $env:AWS_SECRET_ACCESS_KEY
```

⚠ **Set these in the active virtual environment, not the base environment.**

Step 4 — Add to Constants

In `src/constants/__init__.py`:

```
python
```

```
AWS_ACCESS_KEY_ID_ENV_KEY = "AWS_ACCESS_KEY_ID"  
AWS_SECRET_ACCESS_KEY_ENV_KEY = "AWS_SECRET_ACCESS_KEY"  
REGION_NAME = "us-east-1"
```

Step 5 — Create S3 Bucket

Concept: S3 (Simple Storage Service) is AWS's object storage. Buckets are containers; objects are files.

Steps:

1. AWS Console → S3 → **Create bucket**.
2. **Region:** `us-east-1` (must match).
3. **Bucket name:** `my-model-ml-proj` (must be globally unique).
4. **Uncheck "Block all public access"** — for personal projects. In production, keep it blocked and use IAM policies.
5. Acknowledge the warning.
6. Create bucket.

🔌 PART B — AWS Connection Code

File 1 — `src/configuration/aws_connection.py`

Purpose: Establish a connection to S3 using boto3.

```
python
```

```
import boto3  
import os  
from src.constants import AWS_SECRET_ACCESS_KEY_ENV_KEY, AWS_ACCESS_KEY_ID_ENV_KEY, REGION_NAME  
  
class S3Client:  
    s3_client = None  
    s3_resource = None
```

```

def __init__(self, region_name=REGION_NAME):
    if S3Client.s3_resource is None or S3Client.s3_client is None:
        __access_key_id = os.getenv(AWS_ACCESS_KEY_ID_ENV_KEY)
        __secret_access_key = os.getenv(AWS_SECRET_ACCESS_KEY_ENV_KEY)

        if __access_key_id is None:
            raise Exception(f"Environment variable: {AWS_ACCESS_KEY_ID_ENV_KEY} is not set.")

        if __secret_access_key is None:
            raise Exception(f"Environment variable: {AWS_SECRET_ACCESS_KEY_ENV_KEY} is not set.")

    S3Client.s3_resource = boto3.resource(
        "s3",
        aws_access_key_id=__access_key_id,
        aws_secret_access_key=__secret_access_key,
        region_name=region_name
    )
    S3Client.s3_client = boto3.client(
        "s3",
        aws_access_key_id=__access_key_id,
        aws_secret_access_key=__secret_access_key,
        region_name=region_name
    )

    self.s3_resource = S3Client.s3_resource
    self.s3_client = S3Client.s3_client

```

Key Concepts:

- boto3 — AWS SDK for Python.
- **Singleton pattern** — only one S3 client/resource is created and reused.
- **Environment variable validation** — raises clear exceptions if missing.

File 2 — src/aws_storage/s3_storage.py (Cloud Storage)

Purpose: Provide high-level methods for file management, upload, and retrieval in S3.

python

```

import os
import sys
import pickle
import numpy as np
import pandas as pd

```

```

from src.configuration.aws_connection import S3Client
from src.exception import CustomException
from src.logger import logging
from io import StringIO

class SimpleStorageService:
    def __init__(self):
        s3_client = S3Client()
        self.s3_resource = s3_client.s3_resource
        self.s3_client = s3_client.s3_client

    def s3_key_path_available(self, bucket_name, s3_key) -> bool:
        try:
            bucket = self.s3_resource.Bucket(bucket_name)
            file_objects = [file_object for file_object in bucket.objects.filter(Prefix=s
3_key)]

            return len(file_objects) > 0
        except Exception as e:
            raise CustomException(e, sys)

    @staticmethod
    def read_object(object_name, decode=True, **kwargs) -> str:
        try:
            s3_client = S3Client().s3_client
            response = s3_client.get_object(Bucket=kwargs["bucket_name"], Key=object_nam
e)

            object_data = response["Body"].read()
            if decode:
                object_data = object_data.decode("utf-8")
            return object_data
        except Exception as e:
            raise CustomException(e, sys)

    def get_file_object(self, filename, bucket_name) -> list:
        try:
            bucket = self.s3_resource.Bucket(bucket_name)
            file_objects = [file_object for file_object in bucket.objects.filter(Prefix=f
ilename)]

            file_objs = []
            for file_object in file_objects:
                file_obj = self.read_object(file_object.key, decode=False, bucket_name=bu
cket_name)

                file_objs.append(file_obj)
            return file_objs
        except Exception as e:
            raise CustomException(e, sys)

```

```
def load_model(self, model_name, bucket_name):
    try:
        model_file = self.get_file_object(model_name, bucket_name)
        model_obj = pickle.loads(model_file[0])
        return model_obj
    except Exception as e:
        raise CustomException(e, sys)

def upload_file(self, from_filename, to_filename, bucket_name):
    try:
        self.s3_resource.meta.client.upload_file(from_filename, bucket_name, to_filename)
    except Exception as e:
        raise CustomException(e, sys)

def download_file(self, bucket_name, model_name, model_dst_path):
    try:
        self.s3_resource.meta.client.download_file(bucket_name, model_name, model_dst_path)
    except Exception as e:
        raise CustomException(e, sys)
```

Key Methods:

Method	Purpose
s3_key_path_available	Check if a key exists in a bucket
read_object	Read an object from S3 (optionally decode)
get_file_object	Fetch file object(s) as bytes
load_model	Load a pickled model from S3
upload_file	Upload a local file to S3
download_file	Download an S3 object to local path

File 3 — src/entity/s3_estimator.py

Purpose: Wrapper class for saving/loading models to/from S3.

python

```

from src.cloud_storage.aws_storage import SimpleStorageService
from src.exception import CustomException
from src.logger import logging
from src.entity.estimator import MyModel
import sys
from pandas import DataFrame

class Proj1Estimator:
    def __init__(self, bucket_name, model_path):
        self.bucket_name = bucket_name
        self.s3 = SimpleStorageService()
        self.model_path = model_path
        self.loaded_model: MyModel = None

    def is_model_present(self, model_path):
        try:
            return self.s3.s3_key_path_available(bucket_name=self.bucket_name, s3_key=model_path)
        except Exception as e:
            raise CustomException(e, sys)

    def load_model(self) -> MyModel:
        try:
            return self.s3.load_model(self.model_path, bucket_name=self.bucket_name)
        except Exception as e:
            raise CustomException(e, sys)

    def save_model(self, from_file, remove=False) -> None:
        try:
            self.s3.upload_file(from_file, to_filename=self.model_path, bucket_name=self.bucket_name)
            logging.info(f"{from_file} uploaded to S3")
        except Exception as e:
            raise CustomException(e, sys)

    def predict(self, dataframe: DataFrame):
        try:
            if self.loaded_model is None:
                self.loaded_model = self.load_model()
            return self.loaded_model.predict(dataframe=dataframe)
        except Exception as e:
            raise CustomException(e, sys)

```

Key Concepts:

- `is_model_present` → checks if model exists in S3.
- `save_model` → uploads local model to S3.
- `predict` → lazy-loads model and makes predictions.

PART C — Model Evaluation Component

Purpose

Compare the **newly trained model** against the **currently deployed model** (if any) in S3. Accept the new model only if it's sufficiently better.

Constants

python

```
MODEL_EVALUATION_CHANGED_THRESHOLD_SCORE: float = 0.02
MODEL_BUCKET_NAME = "my-model-ml-proj"
MODEL_PUSHER_S3_KEY = "model-registry"
```

- `CHANGED_THRESHOLD_SCORE` = minimum improvement required to accept a new model.
- `MODEL_BUCKET_NAME` = S3 bucket name.
- `MODEL_PUSHER_S3_KEY` = folder in S3 where models live.

Config Entity — `ModelEvaluationConfig`

python

```
@dataclass
class ModelEvaluationConfig:
    changed_threshold_score: float = MODEL_EVALUATION_CHANGED_THRESHOLD_SCORE
    bucket_name: str = MODEL_BUCKET_NAME
    s3_model_key_path: str = MODEL_PUSHER_S3_KEY
```

Artifact Entity — `ModelEvaluationArtifact`

python

```
@dataclass
class ModelEvaluationArtifact:
    is_model_accepted: bool
    changed_accuracy: float
    s3_model_path: str
    trained_model_path: str
```

Component — src/components/model_evaluation.py

python

```
class ModelEvaluation:
    def __init__(self,
                  model_evaluation_config: ModelEvaluationConfig,
                  data_ingestion_artifact: DataIngestionArtifact,
                  model_trainer_artifact: ModelTrainerArtifact):
        try:
            self.model_evaluation_config = model_evaluation_config
            self.model_trainer_artifact = model_trainer_artifact
            self.data_ingestion_artifact = data_ingestion_artifact
        except Exception as e:
            raise CustomException(e, sys)

    def get_best_model(self):
        try:
            bucket_name = self.model_evaluation_config.bucket_name
            model_path = self.model_evaluation_config.s3_model_key_path
            proj1_estimator = Proj1Estimator(bucket_name=bucket_name, model_path=model_path)

            if proj1_estimator.is_model_present(model_path=model_path):
                return proj1_estimator
            return None
        except Exception as e:
            raise CustomException(e, sys)

    def evaluate_model(self) -> ModelEvaluationArtifact:
        try:
            is_model_accepted = True
            logging.info("Entered evaluate_model method of ModelEvaluation class")

            # Load test data
            test_file_path = self.data_ingestion_artifact.test_file_path
            test_df = pd.read_csv(test_file_path)
            x, y = test_df.drop(TARGET_COLUMN, axis=1), test_df[TARGET_COLUMN]
```

```

# Transform test features (same steps as data transformation)
x = map_gender_column(x)
x = drop_column(x, column_name="_id")
x = create_dummy_columns(x)
x = rename_columns(x)

# Load new trained model
trained_model = load_object(file_path=self.model_trainer_artifact.trained_model_file_path)

trained_model_f1_score = self.model_trainer_artifact.metric_artifact.f1_score

best_model_f1_score = None
best_model = self.get_best_model()

if best_model is not None:
    y_hat_best_model = best_model.predict(x)
    best_model_f1_score = f1_score(y, y_hat_best_model)

temp_best_model_score = 0 if best_model_f1_score is None else best_model_f1_score

result = trained_model_f1_score - temp_best_model_score

if result < self.model_evaluation_config.changed_threshold_score:
    is_model_accepted = False
    logging.info(f"Trained model is not accepted as it's not better than the best model")
else:
    logging.info(f"Trained model is accepted as it's better than the best model")

model_evaluation_artifact = ModelEvaluationArtifact(
    is_model_accepted=is_model_accepted,
    changed_accuracy=result,
    s3_model_path=self.model_evaluation_config.s3_model_key_path,
    trained_model_path=self.model_trainer_artifact.trained_model_file_path
)
logging.info(f"Model evaluation artifact: {model_evaluation_artifact}")
return model_evaluation_artifact
except Exception as e:
    raise CustomException(e, sys)

```

Logic Breakdown

Scenario	Result
S3 has no model (first run)	best_model_f1_score = None → temp_best_model_score = 0 → any new model accepted (if f1 > 0)
S3 has a model	Compare trained_f1 - best_f1 against changed_threshold_score (0.02)
New model not better by threshold	is_model_accepted = False
New model better by threshold	is_model_accepted = True

Pipeline Integration

python

```
def start_model_evaluation(self,
                           data_ingestion_artifact: DataIngestionArtifact,
                           model_trainer_artifact: ModelTrainerArtifact) -> ModelEvaluationArtifact:
    try:
        model_evaluation_config = ModelEvaluationConfig()
        model_evaluation = ModelEvaluation(
            model_evaluation_config=model_evaluation_config,
            data_ingestion_artifact=data_ingestion_artifact,
            model_trainer_artifact=model_trainer_artifact
        )
        model_evaluation_artifact = model_evaluation.evaluate_model()
        logging.info(f"Model evaluation completed: {model_evaluation_artifact}")
        return model_evaluation_artifact
    except Exception as e:
        raise CustomException(e, sys)
```



PART D — Model Pusher Component

Purpose

Push the accepted model to S3 **only if** `is_model_accepted == True` .

Artifact — ModelPusherArtifact

python

```
@dataclass
class ModelPusherArtifact:
    bucket_name: str
    s3_model_path: str
```

Config — ModelPusherConfig

python

```
@dataclass
class ModelPusherConfig:
    bucket_name: str = MODEL_BUCKET_NAME
    s3_model_path: str = MODEL_PUSHER_S3_KEY
```

Component — src/components/model_pusher.py

python

```
class ModelPusher:
    def __init__(self,
                  model_pusher_config: ModelPusherConfig,
                  model_evaluation_artifact: ModelEvaluationArtifact):
        try:
            self.model_pusher_config = model_pusher_config
            self.model_evaluation_artifact = model_evaluation_artifact
            self.s3 = SimpleStorageService()
        except Exception as e:
            raise CustomException(e, sys)

    def initiate_model_pusher(self) -> ModelPusherArtifact:
        try:
            logging.info("Entered initiate_model_pusher method of ModelPusher class")

            trained_model_path = self.model_evaluation_artifact.trained_model_path
            model_file_path = self.model_pusher_config.s3_model_path
            bucket_name = self.model_pusher_config.bucket_name

            if self.model_evaluation_artifact.is_model_accepted:
                self.s3.upload_file(
                    from_filename=trained_model_path,
```

```

        to_filename=model_file_path,
        bucket_name=bucket_name
    )
    logging.info(f"Model pushed to S3: {trained_model_path}")
else:
    logging.info("Trained model is not accepted, so not pushed to S3")

    model_pusher_artifact = ModelPusherArtifact(
        bucket_name=bucket_name,
        s3_model_path=model_file_path
    )
    logging.info(f"Model pusher artifact: {model_pusher_artifact}")
    return model_pusher_artifact
except Exception as e:
    raise CustomException(e, sys)

```

Key Logic

- Only pushes if `is_model_accepted == True` .
- Prevents worse models from replacing better deployed ones.

Pipeline Integration

python

```

def start_model_pusher(self,
                        model_evaluation_artifact: ModelEvaluationArtifact) -> ModelPusher
Artifact:
    try:
        model_pusher_config = ModelPusherConfig()
        model_pusher = ModelPusher(
            model_pusher_config=model_pusher_config,
            model_evaluation_artifact=model_evaluation_artifact
        )
        model_pusher_artifact = model_pusher.initiate_model_pusher()
        logging.info(f"Model pusher completed: {model_pusher_artifact}")
        return model_pusher_artifact
    except Exception as e:
        raise CustomException(e, sys)

```



python

```
def run_pipeline(self):
    try:
        data_ingestion_artifact = self.start_data_ingestion()
        data_validation_artifact = self.start_data_validation(data_ingestion_artifact)
        data_transformation_artifact = self.start_data_transformation(
            data_ingestion_artifact, data_validation_artifact)
        model_trainer_artifact = self.start_model_trainer(data_transformation_artifact)
        model_evaluation_artifact = self.start_model_evaluation(
            data_ingestion_artifact, model_trainer_artifact)
        model_pusher_artifact = self.start_model_pusher(model_evaluation_artifact)
    except Exception as e:
        raise CustomException(e, sys)
```

Artifact Structure (Updated)

text

```
artifact/
├── <timestamp>/
│   ├── data_ingestion/
│   ├── data_validation/
│   ├── data_transformation/
│   ├── model_trainer/
│   └── (model_evaluation and model_pusher don't save locally – they push to S3)
```

S3 Bucket Structure:

text

```
my-model-ml-proj/
├── model-registry/
│   └── model.pkl
```

Deep Concept: Why Compare Against S3 Model?

The Problem

Every time you re-run the pipeline, a new model is trained. But **not every new model is better**. Deploying a worse model degrades production performance.

The Solution

1. **Store the best model in S3** (versioned, central).
2. On each pipeline run, **evaluate the new model against the S3 model**.
3. Only **push the new model if it's better by a threshold** (e.g., 2% improvement).
4. Otherwise, keep the old model.

Benefits

- **No regression:** Worse models never reach production.
- **Versioning:** S3 keeps the last accepted model.
- **Automation:** The pipeline decides automatically.
- **Production-grade:** Mirrors real MLOps workflows (model registry, champion/challenger).



Testing the Pipeline

First Run (No Model in S3)

- `best_model = None`
- `temp_best_model_score = 0`
- New model accepted if `f1 > 0`
- Model pushed to S3 

Second Run (Model Already in S3)

- `best_model` loaded from S3
- `best_model_f1_score` computed on test data
- Compare with new model's f1
- Accept only if difference $\geq 2\%$

Pro Tip from the Instructor

To see the comparison in action:

1. **First run:** Reduce `n_estimators` from 200 → 20 (weaker model). Push to S3.
2. **Second run:** Restore `n_estimators = 200` (stronger model). It should be accepted with a clear accuracy difference.

Key Takeaways

1. **Never use root AWS account** — always create IAM users.
2. **Always select the same region** (`us-east-1`) across all AWS resources.
3. **Access keys go into environment variables** — never hardcode.
4. **S3 bucket = remote model registry.**
5. `boto3` = AWS SDK for Python.
6. **Singleton S3 client** = efficient connection reuse.
7. **Model Evaluation compares new vs. deployed** using F1 score.
8. `changed_threshold_score` prevents trivial improvements from triggering a push.
9. **Model Pusher only pushes accepted models** — protects production.
10. **Constants file** = single source of truth for bucket names, paths, thresholds.

Common Mistakes to Avoid

Mistake	Fix
Using root AWS account	Create IAM user
Different regions for S3 vs. code	Always <code>us-east-1</code>
Hardcoding access keys	Use environment variables
Pushing every trained model	Only push if accepted
Forgetting to unblock public access (personal projects)	Uncheck the block option

Mistake**Fix**

Not downloading access key CSV

Download and store safely

**Homework**

1. Set up IAM user + access keys + S3 bucket.
2. Set environment variables in the active venv.
3. Implement `aws_connection.py` , `s3_storage.py` , `s3_estimator.py` .
4. Implement Model Evaluation + Model Pusher components.
5. Run the pipeline twice:
 - First with `n_estimators = 20` (weak model).
 - Then with `n_estimators = 200` (strong model).
6. Verify the second model is accepted and pushed.
7. Push code to Git.

**What's Next**

- **Prediction Pipeline** — load model from S3 and predict on new data.
- **Application** — Flask/FastAPI web app.
- **CI/CD** — GitHub Actions, Docker.
- **Deployment** — AWS EC2/ECS or similar.

"Next video: we'll build the Prediction Pipeline and Application, then jump to CI/CD.
Two more videos and the project is complete end-to-end."

End of Part 5 Notes.



ओके सो लेट्स रिज्यूम द प्रोजे...
TXT 56.32KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Part 6 — Prediction Pipeline + FastAPI Application

Project: Full-fledged Production-Grade ML Project (MLOps series)

Prerequisite: Parts 1–5 (MongoDB → Data Ingestion → Validation → Transformation → Model Trainer → Evaluation → Pusher)

Big Picture — Where We Are Now

Up to the previous video, we completed the **entire training pipeline end-to-end**:

text

MongoDB → Data Ingestion → Data Validation → Data Transformation → Model Trainer → Model Evaluation → Model Pusher (S3) 

The training pipeline is now **complete**. Every time we run it, it executes the full flow and pushes the best model to S3.

But **achieving a trained model is not the end goal** — we need to:

1. **Serve predictions** — let users input data and get predictions.
2. **Deploy** — host it somewhere accessible.

This video covers:

text

Step 25: Create Prediction Pipeline

Step 26: Set up FastAPI Application

Next (final) video: **CI/CD + Deployment**.

1 The Prediction Pipeline

Purpose

Load the trained model from S3 and use it to predict on new user input. It's the **inference counterpart** of the training pipeline.

File — src/pipeline/prediction_pipeline.py

python

```
import os
import sys
import pandas as pd
from src.exception import CustomException
from src.logger import logging
from src.utils.main_utils import load_object
from src.entity.config_entity import VehiclePredictorConfig
from src.entity.s3_estimator import Proj1Estimator

class VehicleData:
    def __init__(self,
                 Gender, Age, Driving_License, Region_Code,
                 Previously_Insured, Vehicle_Age, Vehicle_Damage,
                 Annual_Premium, Policy_Sales_Channel, Vintage):
        self.Gender = Gender
        self.Age = Age
        self.Driving_License = Driving_License
        self.Region_Code = Region_Code
        self.Previously_Insured = Previously_Insured
        self.Vehicle_Age = Vehicle_Age
        self.Vehicle_Damage = Vehicle_Damage
        self.Annual_Premium = Annual_Premium
        self.Policy_Sales_Channel = Policy_Sales_Channel
        self.Vintage = Vintage

    def get_vehicle_input_data_frame(self) -> pd.DataFrame:
        try:
            vehicle_input_dict = {
                "Gender": [self.Gender],
                "Age": [self.Age],
                "Driving_License": [self.Driving_License],
                "Region_Code": [self.Region_Code],
                "Previously_Insured": [self.Previously_Insured],
                "Vehicle_Age": [self.Vehicle_Age],
                "Vehicle_Damage": [self.Vehicle_Damage],
                "Annual_Premium": [self.Annual_Premium],
                "Policy_Sales_Channel": [self.Policy_Sales_Channel],
                "Vintage": [self.Vintage]
```

```
    }
    return pd.DataFrame(vehicle_input_dict)
except Exception as e:
    raise CustomException(e, sys)

class VehicleDataClassifier:
    def __init__(self, prediction_pipeline_config: VehiclePredictorConfig = VehiclePredictorConfig()):
        try:
            self.prediction_pipeline_config = prediction_pipeline_config
        except Exception as e:
            raise CustomException(e, sys)

    def predict(self, dataframe) -> str:
        try:
            logging.info("Entered predict method of VehicleDataClassifier class")
            model = Proj1Estimator(
                bucket_name=self.prediction_pipeline_config.model_bucket_name,
                model_path=self.prediction_pipeline_config.model_file_path
            )
            result = model.predict(dataframe)
            return result
        except Exception as e:
            raise CustomException(e, sys)
```

Key Concepts

Concept	Explanation
VehicleData class	A data container for raw user inputs. Has a method to convert to a Pandas DataFrame.
VehicleDataClassifier class	Wraps the S3 estimator. <code>predict()</code> loads the model and returns predictions.
VehiclePredictorConfig	Holds <code>model_bucket_name</code> and <code>model_file_path</code> (from constants).
Lazy loading	Model is loaded only when <code>predict()</code> is called, not at import time.

Config Entity — VehiclePredictorConfig

```
python
```

```
@dataclass
class VehiclePredictorConfig:
    model_bucket_name: str = MODEL_BUCKET_NAME
    model_file_path: str = MODEL_PUSHER_S3_KEY + "/model.pkl"
```

Why This Design?

- **Separation:** The prediction pipeline knows nothing about the web layer.
- **Reusability:** The same pipeline can be used in a CLI, a batch job, or a web API.
- **S3-backed:** Always uses the latest accepted model from S3.

2 FastAPI Application

Why FastAPI?

- **Fast:** Async-first, high performance.
- **Type-safe:** Uses Python type hints.
- **Auto docs:** Swagger UI at `/docs` out of the box.
- **Modern:** Great for ML serving.

Project Structure

```
text
```

```
project/
├── static/
│   └── css/
│       └── style.css
├── templates/
│   └── vehicle_data.html
├── app.py
└── src/
    └── pipeline/
        └── prediction_pipeline.py
```

File — app.py

python

```
from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import Response
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from starlette.responses import RedirectResponse
from uvicorn import run as app_run
import os
from typing import Optional

from src.pipeline.prediction_pipeline import VehicleData, VehicleDataClassifier
from src.pipeline.training_pipeline import TrainingPipeline
from src.constants import APP_HOST, APP_PORT

app = FastAPI()

# Mount static directory for serving CSS
app.mount("/static", StaticFiles(directory="static"), name="static")

# Setup Jinja2 template engine for rendering HTML
templates = Jinja2Templates(directory="templates")

# CORS – allow all origins for cross-origin resource sharing
origins = ["*"]
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/", tags=["authentication"])
async def index(request: Request):
    return templates.TemplateResponse(
        "vehicle_data.html", {"request": request, "context": "Rendering"}
    )

@app.get("/train")
async def trainRouteClient():
```

```

try:
    train_pipeline = TrainingPipeline()
    train_pipeline.run_pipeline()
    return Response("Training successful !!")
except Exception as e:
    return Response(f"Error Occurred! {e}")

@app.post("/")
async def predictRouteClient(request: Request):
    try:
        form = await request.form()
        Gender = form.get("Gender")
        Age = int(form.get("Age"))
        Driving_License = int(form.get("Driving_License"))
        Region_Code = float(form.get("Region_Code"))
        Previously_Insured = int(form.get("Previously_Insured"))
        Vehicle_Age = form.get("Vehicle_Age")
        Vehicle_Damage = form.get("Vehicle_Damage")
        Annual_Premium = float(form.get("Annual_Premium"))
        Policy_Sales_Channel = float(form.get("Policy_Sales_Channel"))
        Vintage = int(form.get("Vintage"))

        vehicle_data = VehicleData(
            Gender, Age, Driving_License, Region_Code,
            Previously_Insured, Vehicle_Age, Vehicle_Damage,
            Annual_Premium, Policy_Sales_Channel, Vintage
        )
        vehicle_df = vehicle_data.get_vehicle_input_data_frame()
        model_predictor = VehicleDataClassifier()
        value = model_predictor.predict(dataframe=vehicle_df)[0]

        status = "Response Yes" if value == 1 else "Response No"

        return templates.TemplateResponse(
            "vehicle_data.html",
            {"request": request, "context": status}
        )
    except Exception as e:
        return {"status": False, "error": f"{e}"}

if __name__ == "__main__":
    app_run(app, host=APP_HOST, port=APP_PORT)

```

Key Concepts Breakdown

A. FastAPI Application Initialization

```
python
```

```
app = FastAPI()
```

Creates the ASGI application object. This is the entry point for all HTTP requests.

B. Static Files Mounting

```
python
```

```
app.mount("/static", StaticFiles(directory="static"), name="static")
```

- **Static files** = CSS, JS, images that don't change per request.
- Mounting `/static` means `/static/css/style.css` serves from `static/css/style.css`.
- The `name="static"` is used for URL reversing.

C. Jinja2 Templates

```
python
```

```
templates = Jinja2Templates(directory="templates")
```

- **Jinja2** = templating engine for rendering HTML with dynamic data.
- `templates/vehicle_data.html` is the HTML form.
- We render it with variables like `context` (the prediction result).

D. CORS Middleware — Deep Concept

```
python
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["*"],  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],  
)
```

What is CORS (Cross-Origin Resource Sharing)?

- Browsers have a **Same-Origin Policy**: by default, a page loaded from `origin A` cannot make requests to `origin B`.
- **Origin** = protocol + domain + port. E.g., `http://localhost:3000` vs `http://localhost:8000` are different origins.
- If your frontend is on port 3000 and backend on port 8000, the browser blocks the request **unless** the server says "I allow this origin".

Real-world problem: You deploy your app on AWS. The URL loads but the page doesn't work because the browser blocks requests. Adding CORS with `allow_origins=["*"]` fixes it.

CORS flow:

1. Browser sends a **preflight OPTIONS request** to check if the origin is allowed.
2. Server responds with `Access-Control-Allow-Origin` header.
3. If allowed, the actual request proceeds; otherwise, the browser blocks it.

⚠ In production, replace `["*"]` with specific origins (e.g., your frontend domain) for security.

E. Async/Await — Deep Concept

python

```
@app.get("/")
async def index(request: Request):
    ...
```

async and await — the crash course:

- **Synchronous** code: runs one task at a time. Task 2 waits for Task 1 to finish.
- **Asynchronous** code: while Task 1 is waiting (e.g., for user input, DB response, network), the event loop runs Task 2.

Why it matters for a web app:

Imagine two users hit your `/predict` endpoint:

- User A submits a form. Your app is waiting for the ML model to respond.
- **Without async:** User B's request is queued until A's prediction finishes. Slow.
- **With async:** While A's prediction is running, the event loop handles B's request. Fast.

`async def` — declares a coroutine. It can be paused and resumed.

`await` — pauses the current coroutine until the awaited operation completes, letting other coroutines run.

```
python
```

```
form = await request.form()  # pause here; other requests can be served meanwhile
```

Rule of thumb:

- I/O-bound tasks (DB, network, file) → use `async / await`.
- CPU-bound tasks (heavy ML inference) → may block the event loop; use a thread pool or separate service.

F. Routes

Route	Method	Purpose
/	GET	Render the HTML form
/	POST	Handle form submission → prediction
/train	GET	Trigger the full training pipeline

Why two / routes? FastAPI differentiates by HTTP method. `GET /` shows the form; `POST /` processes it.

G. Reading Form Data

```
python
```

```
form = await request.form()
Gender = form.get("Gender")
Age = int(form.get("Age"))
```

- `await request.form()` parses the POST body (multipart/form-data or urlencoded).
- `form.get(...)` extracts individual fields.
- Type conversion (`int` , `float`) is done manually — FastAPI's Pydantic models would do this automatically, but the form-based approach is simpler for HTML forms.

H. Prediction Flow

python

```

vehicle_data = VehicleData(...) # 1. bundle inputs
vehicle_df = vehicle_data.get_vehicle_input_data_frame() # 2. to DataFrame
model_predictor = VehicleDataClassifier() # 3. get predictor
value = model_predictor.predict(dataframe=vehicle_df)[0] # 4. predict
status = "Response Yes" if value == 1 else "Response No" # 5. map result
return templates.TemplateResponse("vehicle_data.html", {"request": request, "context": status})

```

The rendered HTML displays the prediction in the `context` variable.

I. Training Route (Bonus)

python

```

@app.get("/train")
async def trainRouteClient():
    try:
        train_pipeline = TrainingPipeline()
        train_pipeline.run_pipeline()
        return Response("Training successful !!")
    except Exception as e:
        return Response(f"Error Occurred! {e}")

```

- Hitting `/train` triggers the **entire training pipeline** (data ingestion → ... → model pusher).
- **In production**, you'd typically **remove this route** — end users shouldn't trigger retraining.

3 Templates and Static Files

templates/vehicle_data.html

An HTML form with fields matching the model's input features:

- Gender (dropdown)
- Age (number)
- Driving_License (0/1)
- Region_Code (number)

- Previously_Insured (0/1)
- Vehicle_Age (dropdown: < 1 Year , 1-2 Year , > 2 Years)
- Vehicle_Damage (Yes/No)
- Annual_Premium (number)
- Policy_Sales_Channel (number)
- Vintage (number)

The form submits via POST to / , and the context variable displays the prediction.

static/css/style.css

Styling for the form — cleaner layout, better UX.

Instructor's note: The HTML/CSS was generated using ChatGPT by providing the project requirements. Line-by-line it's interpretable, but the instructor isn't a web dev expert — this is a legitimate use of LLMs to scaffold UI.

4 Running the Application

Step 1 — Install FastAPI + Uvicorn

```
bash
```

```
pip install fastapi uvicorn jinja2 python-multipart
```

Step 2 — Add constants

```
python
```

```
APP_HOST = "0.0.0.0"
```

```
APP_PORT = 5000
```

Step 3 — Run

```
bash
```

```
python app.py
```

Or directly with uvicorn:

```
bash

uvicorn app:app --host 0.0.0.0 --port 5000 --reload
```

Step 4 — Access

Open: `http://localhost:5000`

You'll see the form. Fill it in → submit → prediction appears.

Step 5 — Test with real test data

Open `artifact/<timestamp>/data_ingestion/ingested/test.csv` , pick a row, and enter those values into the form. Compare the prediction against the actual label.

5 Common Bugs & Fixes

Bug	Cause	Fix
Cannot import VehiclePredictorConfig	Missing config entity	Add <code>VehiclePredictorConfig</code> to <code>src/entity/config_entity.py</code>
CSS not loading	Wrong static path	Ensure <code>static/css/style.css</code> matches <code>app.mount("/static", ...)</code>
Template not found	Wrong template dir	Ensure <code>templates/vehicle_data.html</code> exists
Page loads but no styling	CORS or path issue	Check browser console for 404s
<code>python-multipart</code> missing	Form parsing needs it	<code>pip install python-multipart</code>

The instructor hit a CSS/template path bug and a missing `VehiclePredictorConfig` — both fixed live.

6 Full Local Project Flow (End-to-End)

text

1. User opens `http://localhost:5000`
2. Sees the HTML form (`templates/vehicle_data.html` + `static/css/style.css`)
3. Fills in vehicle data
4. Submits (POST /)
5. `app.py`:
 - reads form data
 - builds `VehicleData` object
 - converts to `DataFrame`
 - instantiates `VehicleDataClassifier`
 - calls `predict()` → loads model from S3 → predicts
 - maps 0/1 → "Response No"/"Response Yes"
6. Re-renders HTML with the result in ``context``

Training route (GET /train):

text

1. User hits `http://localhost:5000/train`
2. `TrainingPipeline().run_pipeline()` executes:
Data Ingestion → Validation → Transformation → Trainer → Evaluation → Pusher
3. Model pushed to S3

Key Takeaways

1. **Prediction pipeline is separate from training pipeline** — clean separation of concerns.
2. **FastAPI** = modern, fast, async-first Python web framework.
3. **CORS** = browser security mechanism; middleware allows cross-origin requests.
4. `async / await` = concurrency for I/O-bound tasks; keeps the app responsive.
5. **Jinja2 templates** = dynamic HTML rendering.
6. **Static files** = CSS/JS/images served directly.

7. `/train` **route** = convenient for dev, remove in production.
8. **S3-backed predictions** = always uses the latest accepted model.
9. **LLMs can scaffold UI** — legitimate and efficient.
10. **The full project runs locally end-to-end** — training and prediction both work.

Production Considerations

Concern	Recommendation
<code>allow_origins=["*"]</code>	Restrict to your frontend domain in production
<code>/train</code> route exposed	Remove or protect with auth in production
<code>APP_HOST = "0.0.0.0"</code>	Fine for containers; use a reverse proxy (Nginx) in production
Model loading per request	Cache the loaded model in memory
No authentication	Add auth for production
No rate limiting	Add rate limiting to prevent abuse
No logging/monitoring	Add structured logging + metrics

Homework

1. Implement `prediction_pipeline.py` with `VehicleData` and `VehicleDataClassifier`.
2. Add `VehiclePredictorConfig` to config entity.
3. Create `static/` and `templates/` folders with the generated HTML/CSS.
4. Implement `app.py` with FastAPI + CORS + Jinja2.
5. Run `python app.py`, open `http://localhost:5000`.
6. Test predictions using rows from `test.csv`.
7. Test the `/train` route (optional).
8. Push code to Git.

What's Next — Final Video

- **CI/CD pipeline** (GitHub Actions)
- **Docker** containerization
- **Deployment** to a cloud server (AWS EC2/ECS)
- **Public URL** for the app

"Next video is the last one — we'll execute the CI/CD pipeline and smartly publish this application to a deployed server, making it a production-grade application accessible via a public URL."

End of Part 6 Notes.



ओके सो वेलकम बैक फाइनली...
TXT 108.6KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Part 7 (Final) — Docker, CI/CD, ECR, EC2 & Deployment

Project: Full-fledged Production-Grade ML Project (MLOps series) — **FINAL VIDEO**

Prerequisite: Parts 1–6 (MongoDB → Data Ingestion → Validation → Transformation → Model Trainer → Evaluation → Pusher → Prediction Pipeline → FastAPI App)

Big Picture — Where We Are Now

At the end of Part 6, the ML application was **running locally** on `localhost:5000` . But an app limited to your machine has little value for an employer. This final video takes it **over the internet**:

text

Local FastAPI App → Docker Image → AWS ECR → EC2 Instance → Public URL 
(via GitHub Actions CI/CD with self-hosted runner)

Today's agenda (Steps 27 → end):

1. Write Dockerfile + .dockerignore
2. Set up GitHub Actions workflow (CI + CD)
3. Create AWS ECR repository
4. Launch EC2 instance (Ubuntu)
5. Install Docker on EC2
6. Connect GitHub ↔ EC2 via a **self-hosted runner**
7. Add GitHub repository secrets
8. Execute the CI/CD pipeline
9. Open port 5000 on EC2 (Security Group)
10. Access the app publicly
11. (Bonus) Auto-start runner as a service
12. (Bonus) Generate a README via ChatGPT
13. (CRITICAL) Tear down all AWS services to avoid charges



PART A — Dockerfile & .dockerignore

Why Docker?

- Packages your app + dependencies into a single **image**.
- Runs the same way on any machine (your laptop, EC2, Kubernetes).
- Foundation for CI/CD — the image is the deployable artifact.

Dockerfile

```
dockerfile
```

```
FROM python:3.7-slim-buster
```

```
WORKDIR /app
```

```
COPY . /app
```

```
RUN pip install -r requirements.txt
```

```
EXPOSE 5000
```

```
CMD ["python3", "app.py"]
```

Line-by-Line

Line	Meaning
FROM python:3.7-slim-buster	Base image — lightweight Python 3.7 on Debian Buster. <code>slim</code> reduces image size.
WORKDIR /app	Sets the working directory inside the container.
COPY . /app	Copies all project files into <code>/app</code> inside the image.
RUN pip install -r requirements.txt	Installs all Python dependencies.
EXPOSE 5000	Documents that the container listens on port 5000.
CMD ["python3", "app.py"]	The command that runs when the container starts.

Note: The instructor tried a `uvicorn` -based CMD earlier, had issues, and reverted to `python3 app.py` (which internally calls `uvicorn.run` in `app.py`).

.dockerignore

```
text
```

```
artifacts/  
logs/  
venv/  
.git/  
.gitignore  
.dockerignore  
Dockerfile  
__pycache__/  
*.pyc
```

```
.ipynb_checkpoints/  
notebooks/
```

Why .dockerignore ?

- Keeps the image **small** and **clean**.
- Prevents copying local venv, artifacts, logs, git history — none of which are needed at runtime.
- **Smaller image = faster push to ECR = faster deployment = lower cost.**

Key insight: In production, you'd typically **not** copy local artifacts — the app pulls the model from S3 at runtime.

PART B — GitHub Actions Workflow (CI/CD)

File — .github/workflows/main.yaml

⚠️ Must be inside `.github/workflows/`, not `.github/` directly.

yaml

```
name: Deploy Docker Image to EC2 Instance
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - main
```

```
    paths:
```

```
      - "src/**"  
      - "app.py"  
      - "Dockerfile"  
      - "requirements.txt"  
      - "setup.py"  
      - ".github/workflows/**"
```

```
jobs:
```

```
  continuous-integration:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Checkout code  
        uses: actions/checkout@v3
```

- **name:** Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v1
with:
 - aws-access-key-id:** \${{ secrets.AWS_ACCESS_KEY_ID }}
 - aws-secret-access-key:** \${{ secrets.AWS_SECRET_ACCESS_KEY }}
 - aws-region:** \${{ secrets.AWS_DEFAULT_REGION }}

- **name:** Login to Amazon ECR
id: login-ecr
uses: aws-actions/amazon-ecr-login@v1

- **name:** Build, tag, and push image to Amazon ECR
env:
 - ECR_REGISTRY:** \${{ steps.login-ecr.outputs.registry }}
 - ECR_REPOSITORY:** \${{ secrets.ECR_REPO }}
 - IMAGE_TAG:** latest**run:** |
 - docker build -t \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG .
 - docker push \$ECR_REGISTRY/\$ECR_REPOSITORY:\$IMAGE_TAG

continuous-deployment:

needs: continuous-integration

runs-on: self-hosted

steps:

- **name:** Checkout code
uses: actions/checkout@v3

- **name:** Configure AWS credentials
uses: aws-actions/configure-aws-credentials@v1
with:
 - aws-access-key-id:** \${{ secrets.AWS_ACCESS_KEY_ID }}
 - aws-secret-access-key:** \${{ secrets.AWS_SECRET_ACCESS_KEY }}
 - aws-region:** \${{ secrets.AWS_DEFAULT_REGION }}

- **name:** Login to Amazon ECR
id: login-ecr
uses: aws-actions/amazon-ecr-login@v1

- **name:** Pull latest image and run on EC2
env:
 - ECR_REGISTRY:** \${{ steps.login-ecr.outputs.registry }}
 - ECR_REPOSITORY:** \${{ secrets.ECR_REPO }}
 - IMAGE_TAG:** latest**run:** |
 - docker run -d -e AWS_ACCESS_KEY_ID=\${{ secrets.AWS_ACCESS_KEY_ID }} \
 - e AWS_SECRET_ACCESS_KEY=\${{ secrets.AWS_SECRET_ACCESS_KEY }} \

```
-e AWS_DEFAULT_REGION=${{ secrets.AWS_DEFAULT_REGION }} \
-e MONGODB_URL=${{ secrets.MONGODB_URL }} \
-p 5000:5000 \
$ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
```

Deep Concept Breakdown

A. Trigger

```
yaml

on:
  push:
    branches:
      - main
```

Triggers on push to `main` . The `paths` filter ensures CI only runs when relevant files change (avoids unnecessary runs when editing README, etc.).

B. Job 1 — Continuous Integration

Step	Purpose
Checkout	Pulls the latest code into the runner
Configure AWS creds	Uses secrets for auth
Login to ECR	Authenticates Docker with ECR
Build, tag, push	Builds the image and pushes to ECR

Why CI builds the image? So the exact same image that passes tests is deployed — no environment drift.

C. Job 2 — Continuous Deployment

```
yaml

needs: continuous-integration
runs-on: self-hosted
```

- `needs`: — waits for CI to succeed. If CI fails, CD doesn't run.

- runs-on: self-hosted — **not** GitHub's servers, but **your own EC2** (registered as a self-hosted runner).

Why self-hosted? So the deployment happens **on the EC2 itself** — no SSH from GitHub to EC2 required. The runner is the machine.

D. docker run with Environment Variables

yaml

```
docker run -d \
  -e AWS_ACCESS_KEY_ID=${{ secrets.AWS_ACCESS_KEY_ID }} \
  -e AWS_SECRET_ACCESS_KEY=${{ secrets.AWS_SECRET_ACCESS_KEY }} \
  -e AWS_DEFAULT_REGION=${{ secrets.AWS_DEFAULT_REGION }} \
  -e MONGODB_URL=${{ secrets.MONGODB_URL }} \
  -p 5000:5000 \
  $ECR_REGISTRY/$ECR_REPOSITORY:$IMAGE_TAG
```

- -d — detached mode (runs in background).
- -e — passes environment variables into the container. **This is how the container gets AWS + MongoDB credentials.**
- -p 5000:5000 — **port mapping:** host port 5000 → container port 5000.

Why not hardcode credentials in the workflow? Because the workflow file is committed to Git. Secrets belong in GitHub Secrets.

PART C — AWS Setup

Step 1 — Create ECR Repository

ECR = Elastic Container Registry — AWS's Docker Hub.

Steps:

1. AWS Console → search **ECR**.
2. Ensure region = **us-east-1** (must match CI/CD workflow).
3. Create repository → name: `vehicle-proj` .
4. Copy the **ECR URI** (e.g., `123456789.dkr.ecr.us-east-1.amazonaws.com/vehicle-proj`).

Step 2 — Launch EC2 Instance

EC2 = Elastic Compute Cloud — virtual server in the cloud.

Steps:

1. AWS Console → EC2 → **Launch instance**.
2. Name: `vehicle-data-machine`.
3. AMI: **Ubuntu** (free tier eligible).
4. Instance type: **t2.medium** (better for running Docker than t2.micro; costs a few cents/hour).
5. Key pair: **Create new key pair** → name `proj1-key` → download `.pem` file.
6. Storage: **30 GB**.
7. **Launch instance**.
8. Go to Instances → select the instance → **Connect** → use the browser-based SSH.

Step 3 — Install Docker on EC2

Run these commands in the EC2 terminal:

```
bash
```

```
sudo apt-get update -y
sudo apt-get upgrade -y
sudo apt-get install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker $USER
newgrp docker
```

Verify:

```
bash
```

```
docker ps
```

If you see a table (even empty), Docker is working.

PART D — Self-Hosted GitHub Runner

What is a Self-Hosted Runner?

- Normally, GitHub Actions runs jobs on **GitHub's servers** (`runs-on: ubuntu-latest`).
- A **self-hosted runner** means **your own machine** (the EC2) executes the jobs.
- **Why?** So deployment happens on the target server, no SSH needed.

Setup Steps

On GitHub:

1. Repo → **Settings** → **Actions** → **Runners**.
2. **New self-hosted runner** → choose **Linux**.
3. GitHub provides a set of commands — run them on the EC2.

On EC2 (in the runner directory):

```
bash
```

```
# Download
```

```
mkdir actions-runner && cd actions-runner
```

```
curl -o actions-runner-linux-x64-2.x.x.tar.gz -L https://github.com/actions/runner/releases/download/...
```

```
tar xzf ./actions-runner-linux-x64-2.x.x.tar.gz
```

```
# Configure
```

```
./config.sh --url https://github.com/<your-username>/<your-repo> --token <TOKEN>
```

When prompted:

- Runner group: **Enter** (default)
- Runner name: **self-hosted**
- Labels: **Enter** (skip)
- Work folder: **Enter** (skip)

Start the runner:

```
bash
```

```
./run.sh
```

You should see: Listening for Jobs .

Verify on GitHub: Settings → Actions → Runners → should show **Idle** (green).

 **Important:** Keep the "Add new runner" page open in a tab. If you close it, you lose the token. Open the repo in a **new tab** if needed.

Making the Runner Persistent (Production-Grade)

By default, the runner stops if:

- You press Ctrl+C.
- The EC2 reboots.

Solution — install as a systemd service:

```
bash
```

```
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

Now the runner **always runs in the background** and restarts on reboot.

Other useful commands:

```
bash
```

```
sudo ./svc.sh stop      # Stop the service
sudo ./svc.sh uninstall # Remove the service
```



PART E — GitHub Repository Secrets

Go to: Repo → Settings → Secrets and variables → Actions → New repository secret

Add these secrets:

Secret Name	Value
AWS_ACCESS_KEY_ID	From the IAM user's access key CSV
AWS_SECRET_ACCESS_KEY	From the same CSV
AWS_DEFAULT_REGION	us-east-1
ECR_REPO	vehicle-proj
MONGODB_URL	Your MongoDB connection string (only if you want /train route to work)

⚠️ **Secrets cannot be viewed after creation.** If you need to change one, you must create a new one.

PART F — Run the CI/CD Pipeline

Push to Main

```
bash
```

```
git add .  
git commit -m "CI/CD test 1"  
git push origin main
```

Watch the Pipeline

1. Repo → **Actions** tab.
2. Click the running workflow.
3. You'll see two jobs: `continuous-integration` and `continuous-deployment`.
4. CI runs on GitHub's servers, CD runs on your EC2 (self-hosted).

Verify the Image in ECR

AWS Console → ECR → `vehicle-proj` → should show the `latest` image with a recent push timestamp.

Verify the Container on EC2

In the EC2 terminal:

```
bash
```

```
docker ps
```

You should see the container running, mapped to port 5000.

Open Port 5000 on EC2

By default, EC2 blocks inbound traffic except SSH. We must allow port 5000.

Steps:

1. EC2 Console → Instances → select your instance.
2. **Security** tab → click the **Security Group**.
3. **Edit inbound rules** → **Add rule**:
 - Type: Custom TCP
 - Port: 5000
 - Source: Anywhere (0.0.0.0/0)
4. **Save rules**.

Access the App

Get the EC2's **Public IPv4 address** from the Instances page.

Open in browser:

```
text
```

```
http://<EC2-public-ip>:5000
```

You may see a "Not secure" warning — proceed (we haven't set up HTTPS/SSL yet).

The app is now live on the internet! 🎉

Test it:

- Fill in the form → click Predict → get a response.

- Open in incognito → still works (it's not local anymore).

PART G — CRITICAL: Tear Down AWS Services

The instructor's explicit warning: If you leave these running, your bill will keep growing.

Order of Deletion

1. EC2 Instance

- EC2 → Instances → select → **Instance state** → **Stop** → then **Terminate**.
- Termination deletes the instance and stops billing.

2. IAM User (optional but good practice)

- IAM → Users → select `first-proj` → **Delete**.

3. ECR Repository

- ECR → `vehicle-proj` → select the image → **Delete**.
- Then delete the repository itself.

4. S3 Bucket

- S3 → select bucket → **Empty** (delete all objects first).
- Then **Delete bucket**.

5. Verify

- EC2 → Instances → should show "terminated".
- ECR → no repos.
- S3 → no buckets.

Tip: Even if you forget, AWS usually only charges a few cents to a couple of dollars for a short-lived project. But for long-running resources, costs add up fast.

PART H — Bonus: Auto-Generated README

The Trick

Feed ChatGPT:

- The project flow document (`project_flow.txt`)
- Instructions: *"Create a beautiful-looking README for me using the below document and information. The idea is to impress recruiters and visitors by showcasing the tools, techniques, services, and features of this MLOps project."*

ChatGPT generates a complete README with:

- Project overview
- Architecture diagram (text-based)
- Tech stack
- Setup instructions
- Usage
- Screenshots section
- Credits

Why This Matters

- A well-written README is **the first thing recruiters see** on your GitHub.
- It signals professionalism and attention to detail.
- You can iterate on it with the LLM.

Deep Concepts Summary

Docker Concepts

Concept	Meaning
Image	Read-only template with app + dependencies
Container	Running instance of an image
Dockerfile	Recipe to build an image

Concept	Meaning
<code>.dockerignore</code>	Files to exclude from the image
Port mapping	<code>-p host:container</code> forwards traffic
Environment variables	<code>-e KEY=VALUE</code> passes config into container

CI/CD Concepts

Concept	Meaning
CI	Automatically build + test on every push
CD	Automatically deploy the built artifact
Workflow	YAML file defining CI/CD steps
Job	A group of steps running on a runner
Step	A single action or command
Runner	Machine that executes jobs
Self-hosted runner	Your own machine as the runner
Secrets	Encrypted values injected at runtime
needs:	Job dependency (CD waits for CI)

AWS Concepts

Service	Purpose
IAM	Users, roles, permissions
ECR	Docker image registry
EC2	Virtual server
S3	Object storage (model registry)

Service	Purpose
Security Group	Firewall rules for EC2

Everything built from scratch:

- Logging + Exception modules
- Constants, Config, Artifact layers
- 6 ML pipeline components
- Prediction pipeline
- FastAPI web app
- Docker + CI/CD + AWS deployment

⚠ Common Pitfalls (from the instructor's own experience)

Pitfall	Fix
CSS/template path bugs	Verify <code>static/</code> and <code>templates/</code> paths
Missing <code>VehiclePredictorConfig</code>	Add to config entity
Docker image too large	Use <code>slim</code> base + <code>.dockerignore</code>
Runner offline after reboot	Install as systemd service
Forgetting to open port 5000	Add inbound rule in Security Group
Leaving AWS resources running	Tear down everything
Committing secrets	Use GitHub Secrets
Workflow triggering on README edits	Use <code>paths</code> filter

🎓 What This Project Proves

If you can build and deploy this end-to-end, you can confidently:

- Structure a production ML codebase.
- Implement MLOps components (data, model, deployment).
- Work with AWS (IAM, S3, ECR, EC2).

- Write CI/CD pipelines.
- Containerize ML apps.
- Debug real-world issues (the instructor hit many!).

The instructor's challenge: If even one person builds and deploys this app, ping him on LinkedIn — he'll try to respond, review your code, and as a bonus, schedule a call to discuss your career/learning strategy.

Final Words from the Instructor

"I remember I promised on Day 1 that I'd show you a production-grade project. Today that promise is fulfilled. It feels great. Tell me how many of you completed this series and successfully built and deployed the application. Love to hear it back from you."

What's Next (Beyond This Series)

- **Prediction Pipeline** in batch mode (scheduled jobs).
- **Model Monitoring** (data drift, concept drift).
- **Model Retraining** triggers.
- **Kubernetes** for orchestration.
- **MLflow / DVC** for experiment tracking & versioning.
- **Terraform** for infrastructure as code.

But those are for future projects — this one is **complete**.

End of Part 7 (Final) Notes — Project Complete! 🎉



ओके लेट्स स्टार्ट देन टुडे दिस ...
TXT 211.53KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Kubernetes Theory — Part 1 (Distributed Computing, Microservices & Kubernetes Internals)

Series: End-to-End MLOps Playlist

Prerequisite: Docker, Deployment basics, Data Pipelines, OOP

Purpose: Build the *strong foundation* before hands-on Kubernetes (Part 2)

Why This Video Exists

Most Kubernetes tutorials teach K8s from a **DevOps/SRE perspective**. But as an **MLOps Engineer** or **Data Scientist**, you don't need 100% of Kubernetes — you need a **specific subset** that lets you:

- Deploy ML microservices
- Scale inference endpoints
- Orchestrate training/inference pipelines
- Answer interview questions confidently
- Work alongside platform teams

Key insight: The full K8s ecosystem is enormous. Dedicated **Kubernetes Administrators** spend 5–10 years mastering it. You need the **MLOps-relevant subset** — and this video lays that foundation.

The instructor splits Kubernetes into **two parts**:

Part	Focus
Part 1 (this video)	Theory: Distributed Computing, Microservices, K8s internals
Part 2 (next video)	Hands-on: Clusters, pods, deployments, real project

Distributed Computing — The Foundation

The Restaurant Analogy (Intuition First)

Imagine you run a **one-man restaurant**:

- You take orders
- You cook
- You serve
- You bill

At **100 orders/day**, you manage.

At **1,000 orders/day**, you can't. You either:

- Fail customers, or
- Corrupt your own setup trying to do everything.

Solution: Hire chefs. Divide the work. Each chef handles what they can.

At **10,000 orders/day**, you hire 20 chefs. You (the owner) now:

- Take orders
- Distribute work to chefs
- Monitor quality
- Handle a chef going down
- Communicate with customers

This is distributed computing in a nutshell.

Mapping the Analogy to Software

Restaurant	Software
Store/Restaurant	Cluster
Owner/Lead	Lead Server (Master Node)
Chefs	Worker Nodes
Customers	Users / Requests
Orders	Tasks / Workloads

Restaurant	Software
Hiring/firing	Scaling up/down
Quality checks	Health checks
Backup chef	Fault tolerance

Definition

Distributed Computing refers to a system where **multiple computers and nodes work together** to solve a large problem and process data collaboratively. Tasks are divided among nodes using **parallel processing** for faster and more efficient computation.

The ML Example

Suppose you have a **single column** with **A rows** (A = millions/billions) and you want to apply a **log operation** (e.g., for log-normal distribution normalization).

- **One single machine cannot do this** in reasonable time.
- **Solution:** Split the data across worker nodes. Each node computes log for its chunk. Results are combined.

Interview nugget: Log-normal distribution = distribution of a variable whose **log** is normally distributed. Used in normalization.

Key Terms

Term	Meaning
Cluster	A group of interconnected computers/servers working as a single system
Node	A single computer in the cluster
Lead/Master Node	Manages the cluster, assigns tasks, monitors health
Worker Node	Executes assigned tasks

Term	Meaning
Communication	Nodes exchange data/instructions via network protocols
Concurrency	Multiple tasks run simultaneously across nodes
Fault Tolerance	If one node fails, its work is shifted to another

Benefits of Distributed Computing

Benefit	Explanation
Scalability	Divide tasks among machines to handle larger workloads
Fault Tolerance	If one node fails, work shifts to others — no data loss
Improved Performance	Parallel execution = faster results
Cost Efficiency	Use multiple cheaper machines instead of one expensive supercomputer

Challenges of Distributed Computing

These challenges are **exactly why Kubernetes was invented**.

Challenge	Description
Resource Management	Who allocates CPU/memory/storage across machines? How to avoid over/under-utilization?
Scaling	Adding machines is easy; removing them (scale-down) and rebalancing is hard
Communication & Monitoring	Nodes must constantly communicate; network failures, latency, config errors must be tracked

Challenge	Description
Fault Handling	Detecting node failures, recovering lost data, re-routing tasks — while minimizing downtime
Load Balancing	Distributing tasks evenly based on each node's capability (not just equal chunks)
Configuration & Deployment	Manually deploying software across hundreds of nodes is a logistical nightmare
Monitoring & Debugging	Identifying issues across a distributed system is complex

2 Microservices — Why We Need Them

The Monolithic Architecture (What We've Been Doing)

In our previous ML project, we built everything into **one Docker image**:

text

```
[ Data Ingestion → Preprocessing → Feature Engineering → Model Training → Evaluation → Model Serving → UI ]
```

ALL IN ONE CONTAINER

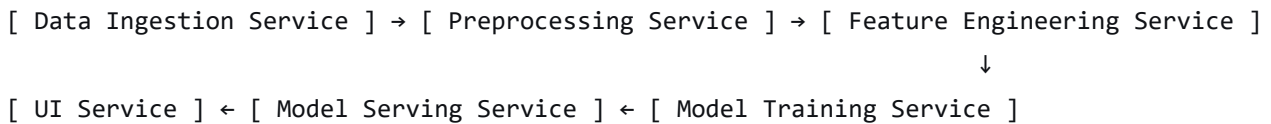
This is called a **Monolithic Architecture**.

The problem: If user traffic spikes, you must scale the **entire monolith** — even though only the **model serving** component is under load. It's like buying a whole outfit (jeans + t-shirt + shoes + watch) just because your shoes wore out.

The Microservices Architecture

Break the monolith into **small, independent services**:

text



Each service:

- Runs **independently**
- Has its **own container**
- Can be **scaled independently**
- Can be **updated independently**
- Communicates via **APIs** (REST, gRPC, messaging)

Real-World Example: Zomato

Zomato is not one giant app. It's many microservices:

- **Restaurant Recommendation Service**
- **Food Recommendation Service** (context-aware: snacks during matches, meals at meal times)
- **Coupon/Discount Service**
- **Delivery Partner Assignment Service**
- **Order Tracking Service**

Each has its own **team**, its own **deployment**, its own **scaling rules**.

Real-World Example: ML in a Bigger App

Our **vehicle insurance prediction** model is one microservice inside a bigger app like **CarDekho** or **Car24**:

- The main app handles listings, search, user accounts, etc.
- Our ML model is a **microservice** that returns **Yes/No** or a price prediction.
- Web developers integrate it into the UI.

Benefits of Microservices

Benefit	Explanation
Independent Scaling	Scale only the service under load (e.g., model serving)
Independent Deployment	Update one service without touching others
Fault Isolation	If one service crashes, others keep running
Technology Freedom	Each service can use a different language/framework
Team Autonomy	Each team owns its service end-to-end
Faster Iteration	Smaller codebases = faster development

Real-Life Analogy: Food Court

- Each **food stall** specializes in one cuisine (dosa, momos, burgers).
- If dosa demand spikes, hire more dosa chefs — not momo chefs.
- If one stall runs out of ingredients, others keep serving.
- A **food court manager** handles electricity, water, ingredients, staffing, and customer flow.

That manager = **Kubernetes**.

How Microservices + Docker + Kubernetes Connect

- **Docker** = packages each microservice into a portable container ("takeout box").
- **Kubernetes** = orchestrates all these containers ("food court manager").
- **Microservices** = the architecture; Docker & K8s = the tools to implement it.

MLOps Relevance: Your ML pipeline components (data ingestion, preprocessing, training, serving) are naturally microservices. Kubernetes lets you scale each independently.

3 Kubernetes Internals — The Core Components

Now we enter **Kubernetes language**. The lead node is called the **Control Plane**. Worker nodes stay worker nodes.

A. Control Plane (The Brain)

Definition: The Control Plane is the **master node** and the **brain** of the Kubernetes cluster. It oversees the entire system, manages workloads, and ensures everything runs as expected.

Think of it as the **factory manager** who delegates tasks and monitors operations.

B. Components of the Control Plane

1. Resource Manager

Definition: Ensures cluster resources (CPU, memory, storage) are allocated efficiently.

How it works:

- You tell the Control Plane: "Deploy this image."
- Resource Manager checks each Worker Node for **available CPU/memory/storage**.
- If a node has room, it schedules a **Pod** there.
- If a node is overloaded or the pod fails, it moves the pod to another node.

Analogy: A **warehouse manager** distributing raw materials to production lines.

2. API Server

Definition: The **interface** between users and the Control Plane. Users interact with Kubernetes through the API Server.

- Acts like a **receptionist** at an office.
- Routes requests to the appropriate component (e.g., a training request goes to the training pipeline; a prediction request goes to model serving).
- **Only entry point** for external communication.

3. etcd (The Cluster Database)

Definition: Kubernetes' **central database** storing all cluster data — including the **current state** of the system (working/failing) and the **desired configuration**.

What gets stored:

- Which pods are running where
- Task assignments to nodes
- Success/failure records
- Configuration state

Analogy: A library catalog — every time you check out or return a book, the catalog is updated.

Interview tip: `etcd` is a **key-value store**. It's the single source of truth for the cluster's state.

C. Worker Nodes (The Muscle)

Definition: Worker nodes run your applications and handle tasks assigned by the Control Plane.

Analogy: Factory workers executing tasks given by the manager.

D. Components of Worker Nodes

1. Kubelet

Definition: An **agent** running on each worker node that ensures containers (your applications) are running as expected. It's like a **shift supervisor** in a factory.

Responsibilities:

- Runs health checks on containers
- Reports failures/high latency back to the Control Plane
- Control Plane then asks Resource Manager to spin up a new pod if needed

2. Kube-proxy

Definition: Manages **network traffic** for pods.

Responsibilities:

- Routes traffic between pods
- Enables pod-to-pod communication

- Enables pod-to-outside-world communication
- Handles **load balancing** across pods
- Example: If pod A finished 50 tasks but pod B only 20, kube-proxy helps redistribute remaining work

Analogy: A **traffic cop** at a busy intersection directing cars (data packets) to their destinations.

3. Pods

Definition: The **smallest deployable unit** in Kubernetes. A pod wraps **one or more containers**.

- Usually one container per pod.
- Can have multiple containers (e.g., data ingestion + preprocessing in one pod).
- **Analogy:** A **shipping container** holding one or more goods.

Full flow:

text

Pod (smallest unit) → contains Container(s) → contains your Docker image/app

4. Shared DB / Volumes

Definition: Shared storage across worker nodes, divided among pods.

- The cluster has a **total storage volume** (e.g., 100 GB).
- Divided among worker nodes/pods as needed.
- Pods can save and share data via this storage.

Analogy: A **shared locker room** where workers leave and access tools/notes.

5. Kubectl

- **Command-line tool** to interact with the Kubernetes cluster.
- We'll use this in Part 2 for hands-on work.

6. Kubemanifest (YAML)

Definition: Configuration files written in YAML that define **what you want Kubernetes to do**.

Analogy: A blueprint for building a house. Kubernetes reads it to know exactly what to construct.

Example:

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: data-ingestion
spec:
  replicas: 3
  selector:
    matchLabels:
      app: data-ingestion
  template:
    metadata:
      labels:
        app: data-ingestion
    spec:
      containers:
      - name: data-ingestion
        image: myrepo/data-ingestion:latest
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
```

7. Service

Definition: Provides a **stable network endpoint** for a set of pods.

The problem it solves:

- Pods come and go (crash, restart, get replaced).
- Their IPs change constantly.
- You don't want to reconfigure inbound rules for thousands of nodes.

The solution:

- A Service gives a **single stable endpoint** (IP/DNS) that routes to whatever pods are currently running that service.
- Even if pods are replaced, the Service endpoint remains the same.

Analogy: A restaurant hotline — no matter who answers the phone, your order is taken. You don't need to know which chef is working.

8. Namespace

Definition: Virtual clusters within a Kubernetes cluster that help organize and isolate resources.

Why?

- Big companies run **multiple projects** in the same physical cluster.
- You don't want Gmail's resources conflicting with Google Drive's.
- Namespaces keep resources logically isolated.

Analogy: An office with departments (HR, Sales, IT) — same building, separate areas.

text

Physical Cluster

- ├ Namespace: gmail
- ├ Namespace: gdrive
- ├ Namespace: gcalendar
- └ Namespace: youtube

9. Replica Set

Definition: Ensures a **specified number of identical pods** are always running.

Why?

- If you want 2 replicas of your data ingestion service for reliability, a ReplicaSet guarantees 2 are always up.
- If one crashes, ReplicaSet spins up a new one.

Analogy: A backup generator — ensuring there's always power, even if one fails.

10. Scheduler

Definition: Decides **which worker node** will run a new pod, based on resource availability.

Flow:

1. A new pod needs to be created.
2. Resource Manager informs the Scheduler.
3. Scheduler picks the best worker node (based on CPU/memory/labels).
4. Kubelet on that node creates and runs the pod.

Connecting the Dots — The Complete Picture

How Kubernetes Solves Distributed Computing Challenges

Challenge	Kubernetes Solution
Resource Management	Resource Manager allocates CPU/memory/storage
Scaling	ReplicaSet + Scheduler scale up/down automatically
Communication	Kube-proxy + API Server + Service handle networking
Fault Handling	Kubelet detects failures → Control Plane → new pod
Load Balancing	Kube-proxy + Scheduler distribute traffic
Configuration/Deployment	Kubemanifest (YAML) declaratively defines everything
Monitoring/Debugging	Kubelet + Control Plane report health to etcd

The Analogy Recap

Without K8s	With K8s
Manually manage chefs, waiters, suppliers for each branch	Central management system monitors every branch in real time
Branch runs out of ingredients → chaos	Auto-restocks ingredients
Staff shortage → slow service	Auto-hires temp staff
One branch crowded, another empty	Redirects customers to less crowded branches
Inefficient, error-prone	Efficient, automated, scalable

Kubernetes is essentially the "central management system" for distributed systems.

5 Microservices + Docker + Kubernetes — The Trio

Layer	Role	Analogy
Microservices	Architectural pattern — break app into small services	Different food stalls
Docker	Package each service into a portable container	Takeout boxes for each stall
Kubernetes	Orchestrate all containers	Food court manager

Why They Go Together

- **Microservices without Docker:** Dependency hell across environments.
- **Docker without Kubernetes:** You manually run `docker run` on each machine.
- **Kubernetes without Docker:** Can't package or isolate services cleanly.
- **All three together:** Portable, scalable, fault-tolerant, production-grade.

Benefits for ML Specifically

- **Independent scaling:** Scale only model serving when inference traffic spikes.

- **Fault isolation:** If preprocessing fails, serving keeps working.
- **Rolling updates:** Update model version without downtime.
- **Resource efficiency:** Right-size CPU/memory per service.
- **Reproducibility:** Same Docker images, same K8s manifests → same behavior everywhere.

6 Key Vocabulary Cheat Sheet

Term	One-Line Definition
Distributed Computing	Multiple machines working together on a large problem
Cluster	Group of interconnected computers acting as one system
Node	A single computer in the cluster
Control Plane	The brain — manages the cluster
Worker Node	The muscle — runs applications
Resource Manager	Allocates CPU/memory/storage
API Server	Interface between users and the cluster
etcd	Cluster's central database (key-value store)
Kubelet	Agent on each worker node; ensures containers run
Kube-proxy	Manages pod networking and load balancing
Pod	Smallest deployable unit; wraps container(s)
Service	Stable network endpoint for a set of pods
Namespace	Virtual cluster within a cluster
ReplicaSet	Ensures N identical pods are always running

Term	One-Line Definition
Scheduler	Decides which node runs a new pod
Kubemanifest	YAML file defining desired state
KubectI	CLI tool to interact with the cluster
Monolithic	Single, large application — all components bundled
Microservices	Small, independent services communicating via APIs

7 Key Takeaways

1. **Distributed computing** solves the "one machine can't do it" problem by dividing work.
2. **It has challenges** — resource management, scaling, communication, fault handling, load balancing, deployment, monitoring.
3. **Kubernetes was created to solve these challenges** — it's the "central management system."
4. **Microservices** break monoliths into small, independently scalable services.
5. **Docker packages microservices; Kubernetes orchestrates them.**
6. **Control Plane = brain** (Resource Manager, API Server, etcd).
7. **Worker Nodes = muscle** (Kubelet, Kube-proxy, Pods, Volumes).
8. **Pods are the smallest deployable unit** — they wrap containers.
9. **Services provide stable endpoints** even as pods come and go.
10. **Namespaces isolate resources** within a cluster.
11. **You don't need to master K8s admin** — only the MLOps-relevant subset.
12. **The notes for this video are available** in the instructor's GitHub repo (link in video description).

8 What's Next — Part 2 (Hands-On)

In the next video, we'll:

- Create an actual Kubernetes cluster
- Write Kubemanifests (YAML) for our ML project
- Deploy data ingestion, preprocessing, training, serving as separate pods
- Set up Services, ReplicaSets, Namespaces
- Use `kubectl` to manage everything
- Experience the "magic" of auto-scaling and self-healing

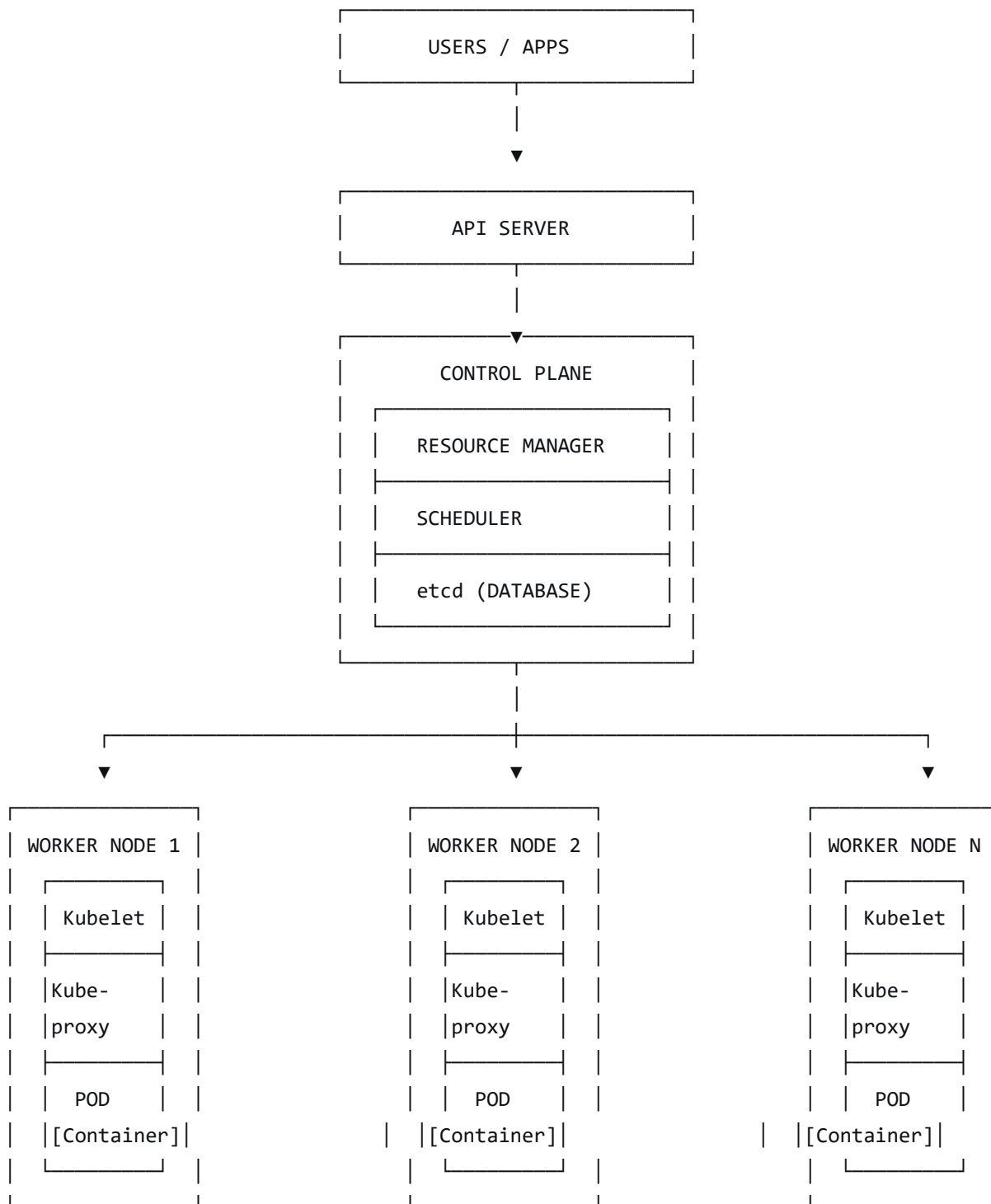
Instructor's note: "Take your time with Part 1. Get 100% confident on these concepts before Part 2. Once you complete Part 2, you'll be a **hero in Kubernetes** within the MLOps field."

9 Interview Prep — Likely Questions

1. What is distributed computing? Give an ML example.
2. What are the benefits and challenges of distributed computing?
3. What is a monolithic vs. microservices architecture? When would you use each?
4. How do Docker and Kubernetes relate to microservices?
5. What is the Control Plane? Name its components.
6. What is etcd? Why is it important?
7. What is a Pod? How is it different from a container?
8. What is a Service in Kubernetes? Why do we need it?
9. What is a Namespace? Give a real-world use case.
10. What is a ReplicaSet? How does it ensure high availability?
11. What does the Scheduler do?
12. How does Kubernetes handle fault tolerance?
13. What is the role of Kubelet and Kube-proxy?
14. How would you deploy an ML model as a microservice on Kubernetes?
15. What is a Kubemanifest? Show a simple example.

10 The Big Picture (One Diagram)

text



Final Notes

- **Don't panic** at the number of components. You only need **understanding**, not memorization.

- The **notes** are in the instructor's **GitHub repo** — clone and refer to them later.
- **Part 2** will make everything **click** as you deploy real services.
- **The goal:** Be interview-ready on Kubernetes for MLOps roles.
- **The promise:** By end of Part 2, you'll be a "hero in Kubernetes" within the MLOps field.

End of Kubernetes Theory — Part 1 Notes.



ओके सो हेलो एंड वेलकम टू ...
TXT 225.48KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Kubernetes Theory — Part 2 (Hands-On: Minikube, Deployments, Services & Load Balancing)

Series: End-to-End MLOps Playlist

Prerequisite: Kubernetes Part 1 (Distributed Computing, Microservices, K8s Internals), Docker, Basic Python/Flask, CI/CD basics

Purpose: Take the theory from Part 1 and **implement it practically** on your local machine

Big Picture — What We're Building Today

In Part 1, we learned the **concepts**: clusters, control plane, worker nodes, pods, services, deployments. In Part 2, we **implement them hands-on**:

text

1. Build a simple Flask app → test locally
2. Dockerize it (Dockerfile → image)
3. Install Minikube → create a local Kubernetes cluster
4. Load the Docker image into the cluster
5. Write Kubernetes manifests (deployment.yaml + service)
6. Deploy → verify pods, services, load balancing

7. Test with Postman
8. Understand how this maps to production (EKS, AKS, GKE)

Instructor's Note on RAM: The video was recorded on a **24 GB RAM** machine, but **12–16 GB is enough** to follow along. If you have **8 GB**, you can still do it — just close other apps. If RAM is a hard constraint, watch the video fully first, then attempt it separately.

1 Why a Simple Flask App (Not an ML App)?

The instructor deliberately uses a **simple Flask app** instead of an ML application because:

- ML apps are **heavy** (dependencies, model loading, memory).
- The **focus of this video is Kubernetes**, not ML.
- A lightweight app keeps the cluster responsive on a laptop.

The App:

- Takes a **name** as input.
- Returns: `Hello <name>, Welcome to Kubernetes Test Application .`
- Uses **Flask + HTML/CSS** (UI generated via an LLM).

Key Insight: Whether you deploy a Flask app or a heavy ML app, the **Kubernetes deployment process is identical**. Once you master the process, swap in your ML app.

2 Step-by-Step Walkthrough

Step 1 — Create the GitHub Repo

- Repo name: `youtube-kubernetes` (or anything descriptive).
- Set it **public** so others can learn from it.
- Add `README`, `.gitignore` (Python), and a license.

Clone locally:

```
bash
```

```
git clone <repo-url>
cd youtube-kubernetes
code .
```

Step 2 — Build the Flask App

app.py (simplified):

```
python

from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    result = None
    if request.method == "POST":
        name = request.form.get("name")
        result = f"Hello {name}, Welcome to Kubernetes Test Application"
    return render_template("index.html", result=result)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Folder structure:

```
text

├─ app.py
├─ templates/
│   └─ index.html
├─ static/
│   └─ css/
│       └─ style.css
├─ Dockerfile
├─ deployment.yaml
└─ requirements.txt
```

Test locally:

```
bash
```

```
python app.py
# Open http://localhost:5000
```

Step 3 — Create the Dockerfile

```
dockerfile

FROM python:3.9-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY . /app

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 5000

CMD ["python", "app.py"]
```

Key points:

Instruction	Purpose
python:3.9-slim	Lightweight base image
PYTHONDONTWRITEBYTECODE=1	Prevents .pyc files → smaller image
PYTHONUNBUFFERED=1	Logs flush immediately (better debugging)
WORKDIR /app	Sets working directory inside container
COPY . /app	Copies app code
RUN pip install ...	Installs dependencies
EXPOSE 5000	Documents the port
CMD ["python", "app.py"]	Runs the app

Build the image:

```
bash
```

```
docker build -t kubernetes-test-app:latest .
```

Verify:

```
bash
```

```
docker images
```

```
# or check Docker Desktop → Images
```

Run the image locally (sanity check):

```
bash
```

```
docker run -p 5000:5000 kubernetes-test-app:latest
```

```
# Open http://localhost:5000
```

Test:

1. Enter a name → Submit → See Hello <name>, Welcome to Kubernetes Test Application .
2. Stop the container → app becomes unavailable.

Image size note: This simple app $\approx$ **130 MB**. An ML app can be **700 MB – 1+ GB**. Keep images small for faster deployments.

Step 4 — Install Minikube

What is Minikube?

Minikube is a tool that **runs a single-node Kubernetes cluster on your local machine**. It's the easiest way to learn Kubernetes without provisioning real servers.

Important: Minikube is for **learning/development** — NOT production. Production uses **managed Kubernetes services** (EKS, AKS, GKE) or on-premises clusters.

Installation (Windows/PowerShell as Administrator):

1. Visit the Minikube install page.
2. Download and run the PowerShell installer.
3. **Restart your system** after installation.

Start the cluster:

```
bash
```

```
minikube start
```

Expected output:

```
text
```

```
* minikube v1.x.x on Windows ...
```

```
* Using the docker driver
```

```
* Starting control plane node minikube in cluster minikube
```

```
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

Check status:

```
bash
```

```
minikube status
```

```
# Should show: host: Running, kubelet: Running, apiserver: Running, kubeconfig: Configured
```

Step 5 — Troubleshooting Minikube Startup Issues

If minikube start fails with "Failed to connect" or proxy errors:

Fix 1 — Disconnect any proxy:

```
powershell
```

```
netsh winhttp reset proxy
```

Fix 2 — Verify network access to Minikube's registry:

```
bash
```

```
nslookup k8s.gcr.io
```

Fix 3 — Reset Minikube:

```
bash
```

```
minikube stop
minikube delete --all
# Restart your system
minikube start
```

Fix 4 — If behind a proxy, pass proxy to Minikube:

```
bash
```

```
minikube start --docker-env HTTP_PROXY=http://proxy:port --docker-env HTTPS_PROXY=http://
proxy:port
```

Step 6 — Explore the Cluster with kubectl

kubectl is the CLI to interact with any Kubernetes cluster (Minikube, EKS, AKS, GKE).

Commands are identical across environments.

Core commands:

```
bash
```

```
# Get all resources in the cluster
```

```
kubectl get all
```

```
# Get pods (only the internal K8s components initially)
```

```
kubectl get pods
```

```
# Get nodes (only control-plane node in Minikube by default)
```

```
kubectl get nodes
```

```
# Get namespaces
```

```
kubectl get namespaces
```

```
# Get services
```

```
kubectl get services
```

What you'll see initially:

Pods like etcd-minikube , kube-apiserver-minikube , kube-controller-manager-minikube , kube-scheduler-minikube — these are the **Kubernetes internal components** running as microservices inside Docker containers.

Key Insight: Kubernetes itself runs as microservices on Docker. That's why "Docker & Kubernetes" are always mentioned together.

List images inside the Minikube cluster:

```
bash
```

```
minikube image list
```

This shows the internal K8s images, but **NOT your app image yet** — you must load it explicitly.

Step 7 — Add More Nodes (Optional)

By default, Minikube creates **one node** (the control plane). To add worker nodes:

```
bash
```

```
minikube start --nodes 3
```

Then:

```
bash
```

```
kubectl get nodes  
# Should show 3 nodes
```

Note: On 8 GB RAM, stick to one node. On 16+ GB, 2–3 nodes are fine.

Step 8 — Load Your Docker Image into Minikube

Since the Minikube cluster runs in its own Docker environment, it **can't see your local Docker images** by default. You must load them:

```
bash
```

```
minikube image load kubernetes-test-app:latest
```

Verify:

```
bash
```

```
minikube image list
# Should now show kubernetes-test-app:latest
```

Alternative (Production): Push your image to a registry (Docker Hub, AWS ECR) and let the cluster pull it. More on this later.

Step 9 — Write the Kubernetes Manifest (deployment.yaml)

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: kubernetes-test-app
  labels:
    app: kubernetes-test-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: kubernetes-test-app
  template:
    metadata:
      labels:
        app: kubernetes-test-app
    spec:
      containers:
        - name: kubernetes-test-app
          image: kubernetes-test-app:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
          resources:
            limits:
              memory: "64Mi"
              cpu: "200m"
---
apiVersion: v1
kind: Service
metadata:
  name: kubernetes-test-app-service
spec:
  selector:
    app: kubernetes-test-app
  ports:
```

```
- protocol: TCP
  port: 80
  targetPort: 5000
type: NodePort
```

Deep Dive — Deployment Manifest:

Field	Meaning
apiVersion: apps/v1	API version for Deployments
kind: Deployment	This resource is a Deployment
metadata.name	Name of the deployment
spec.replicas: 2	Always keep 2 pods running
spec.selector	Matches pods with label app: kubernetes-test-app
spec.template	Pod template (blueprint for pods)
containers.image	Docker image to run
imagePullPolicy: Never	Don't pull from remote registry — use the local image we loaded
containerPort: 5000	Port the container listens on
resources.limits	Max CPU/memory the container can use

imagePullPolicy options:

Value	Behavior
Always	Always pull from registry
IfNotPresent	Pull only if not cached locally
Never	Never pull — rely on local image

If using Docker Hub/ECR: Set `imagePullPolicy: IfNotPresent` and replace `image:` with `yourusername/kubernetes-test-app:latest` .

Deep Dive — Service Manifest:

Field	Meaning
kind: Service	This is a Service resource
spec.selector	Routes traffic to pods with label app: kubernetes-test-app
spec.ports.port: 80	Service listens on port 80
spec.ports.targetPort: 5000	Forwards traffic to container port 5000
type: NodePort	Exposes the service on each node's IP at a static port

Service Types:

Type	Use Case
ClusterIP (default)	Internal-only access
NodePort	Exposes on each node's IP at a port (30000–32767)
LoadBalancer	Cloud provider provisions a load balancer (EKS/AKS/GKE)
ExternalName	Maps service to a DNS name

Step 10 — Apply the Deployment

From the project directory (where `deployment.yaml` lives):

```
bash

kubectl apply -f deployment.yaml
```

Expected:

```
text

deployment.apps/kubernetes-test-app created
```

```
service/kubernetes-test-app-service created
```

Verify pods:

```
bash
```

```
kubectl get pods  
# Should show 2 pods running: kubernetes-test-app-xxxxx
```

Verify deployment:

```
bash
```

```
kubectl get deployments  
# READY 2/2
```

Verify services:

```
bash
```

```
kubectl get services  
# kubernetes-test-app-service NodePort ...
```

Step 11 — Test Self-Healing

Delete a pod manually:

```
bash
```

```
kubectl delete pod <pod-name>
```

What happens?

- Kubernetes **immediately** spins up a new pod to maintain replicas: 2 .
- This is the **magic of Kubernetes** — self-healing in seconds.

You never do this manually in production. It was a demo to show self-healing.

Step 12 — Access the Service

In Minikube:

```
bash
```

```
minikube service kubernetes-test-app-service
```

This opens the app in your browser and shows the URL (e.g., `http://127.0.0.1:64806`).

Note: The port changes every time you restart the Minikube service. Update Postman/scripts accordingly.

Step 13 — MiniKube Dashboard

```
bash
```

```
minikube dashboard
```

Opens a **web UI** showing:

- **Deployments:** 1 deployment
- **Pods:** 2 running
- **ReplicaSets:** 1
- **Services:** 1

Note: This dashboard is a **Minikube-specific learning tool**. In production, **Prometheus + Grafana** are used for monitoring/dashboards.

Step 14 — Check Endpoints

```
bash
```

```
kubectl get endpoints  
# Shows the IP:port for each pod backing the service
```

Also:

```
bash
```

```
kubectl get services
```

- **Internal endpoints** = how pods communicate within the cluster.
- **External endpoints** = how the app is exposed to the outside world (Ingress/Egress).

Ingress vs Egress:

- **Ingress:** Traffic coming **into** the cluster.
- **Egress:** Traffic going **out** of the cluster.

These are advanced concepts — not needed for basic MLOps work, but good to know.

Step 15 — Load Balancing Demo with Postman

What is Postman?

A tool for testing APIs. Download from postman.com.

Test the service manually first:

1. New → HTTP request
2. Method: POST
3. URL: the Minikube service URL (e.g., `http://127.0.0.1:64806`)
4. Body → form-data → key= name , value= Postman User
5. Send → Should return Hello Postman User, Welcome to Kubernetes Test Application

Now test load balancing:

1. Open **two terminals** side by side.
2. Stream logs from each pod:

```
bash
```

```
kubectl logs -f <pod-1-name>
```

```
kubectl logs -f <pod-2-name>
```

3. In Postman: **Collections** → **Run** → **Performance Test**

4. Configure:

- **Load Profile:** Fixed
- **Users:** 5–10
- **Duration:** 1 minute

5. Run → Watch logs in both terminals.

What you'll observe:

- Requests are **distributed across both pods** automatically.
- This is **Kubernetes load balancing** in action.

Key Insight: You don't write any load-balancing rules. Kubernetes does it **automatically** based on pod availability and capacity.

3 How This Maps to Production (EKS, AKS, GKE)

The CI/CD + Kubernetes Flow

In the previous MLOps project, we built:

text

GitHub → GitHub Actions → Build Docker image → Push to ECR → Pull on EC2 → Run

With Kubernetes, the flow changes:

text

GitHub → GitHub Actions → Build Docker image → Push to ECR

↓

kubectl apply -f deployment.yaml (deploys to K8s cluster)

↓

Kubernetes pulls image from ECR → runs in pods

Changes required in `.github/workflows/main.yaml` :

1. Keep the build & push to ECR steps.
2. **Replace** the "SSH to EC2 and `docker run` " step with `kubectl apply` against the Kubernetes cluster.

Sample GitHub Actions snippet:

yaml

```
- name: Deploy to Kubernetes
  uses: azure/setup-kubectl@v3
  with:
    version: 'latest'
```

```
- name: Apply Deployment
  run: |
    kubectl apply -f deployment.yaml
  env:
    KUBECONFIG: ${ secrets.KUBECONFIG }
```

Instructor's Note: We won't implement managed Kubernetes in this playlist (cost reasons), but the concepts carry over.

Managed Kubernetes Services

Provider	Service Name	Notes
AWS	EKS (Elastic Kubernetes Service)	Most popular, integrates with ECR/S3
Azure	AKS (Azure Kubernetes Service)	Beginner-friendly, highly automated
GCP	GKE (Google Kubernetes Engine)	Google invented K8s, very mature

Benefits of managed services:

- Cloud provider handles the **control plane** (no need to hire a K8s admin).
- Auto-scaling, networking, monitoring built-in.
- Integrates seamlessly with other cloud services (S3, RDS, etc.).

On-Premises Kubernetes

- Used by **banks, government institutions** with strict data compliance.
- Tools: **kubeadm, Rancher, k3s**.
- You manage the **entire control plane** yourself.
- Requires dedicated **Kubernetes administrators** (₹50–60 lakh/year salaries).

Deep Concept Recap

Why `imagePullPolicy: Never` ?

The Minikube cluster can't pull from a remote registry (unless you push there). By setting `Never`, we force it to use the **locally loaded** image via `minikube image load`.

For production:

- Push image to ECR/Docker Hub.
- Set `imagePullPolicy: IfNotPresent` OR `Always`.
- Specify full image path: `123456789.dkr.ecr.us-east-1.amazonaws.com/my-app:latest`.

Why replicas: 2?

- **Fault tolerance:** If one pod fails, the other still serves traffic.
- **Load balancing:** Traffic is distributed across pods.
- **Rolling updates:** K8s can update one pod at a time without downtime.

Why a Service?

Pods are **ephemeral** — they get new IPs on restart. A Service provides a **stable endpoint** that routes to whichever pods are currently running. Without a Service, you'd have to reconfigure clients every time a pod restarts.

ReplicaSet vs Deployment

- **ReplicaSet:** Ensures N identical pods are running.
- **Deployment:** Manages ReplicaSets; supports **rolling updates** and **rollbacks**.

You **almost always** use Deployments, not bare ReplicaSets.

5 Key Takeaways

1. **Minikube** = local single-node Kubernetes cluster for learning.
2. **kubectl** commands are **identical** across all K8s environments.
3. **Kubernetes runs as microservices on Docker** — that's why they're mentioned together.
4. **Load image into Minikube** with `minikube image load` before deploying.
5. **Deployment.yaml** defines: image, replicas, resources, ports.

6. **Service** provides stable networking + load balancing across pods.
7. **Self-healing**: Delete a pod → K8s spins up a new one in seconds.
8. **Load balancing is automatic** — no rules needed.
9. **CI/CD + K8s**: Replace "SSH + docker run" with "kubectl apply".
10. **Production K8s** = managed services (EKS, AKS, GKE) or on-prem (kubeadm, Rancher).
11. **Minikube dashboard** is for learning; production uses **Prometheus + Grafana**.
12. **Ingress** = traffic in; **Egress** = traffic out.

6 Command Cheat Sheet

bash

Minikube

minikube start	<i># Start cluster</i>
minikube start --nodes 3	<i># Start with 3 nodes</i>
minikube status	<i># Check cluster status</i>
minikube stop	<i># Stop cluster</i>
minikube delete --all	<i># Delete cluster</i>
minikube image load <image>:<tag>	<i># Load local image into cluster</i>
minikube image list	<i># List images in cluster</i>
minikube service <service-name>	<i># Open service URL in browser</i>
minikube dashboard	<i># Open dashboard</i>

kubectl

kubectl get all	<i># All resources</i>
kubectl get pods	<i># List pods</i>
kubectl get nodes	<i># List nodes</i>
kubectl get services	<i># List services</i>
kubectl get deployments	<i># List deployments</i>
kubectl get namespaces	<i># List namespaces</i>
kubectl get endpoints	<i># List service endpoints</i>
kubectl apply -f deployment.yaml	<i># Apply manifest</i>
kubectl delete -f deployment.yaml	<i># Delete resources</i>
kubectl delete pod <pod-name>	<i># Delete a pod</i>
kubectl logs -f <pod-name>	<i># Stream pod logs</i>
kubectl describe pod <pod-name>	<i># Detailed pod info</i>

Docker

docker build -t <image>:<tag> .	<i># Build image</i>
---------------------------------	----------------------

```
docker images # List images
docker run -p <host>:<container> <image> # Run container
```

7 Common Issues & Fixes

Issue	Fix
minikube start fails with proxy error	netsh winhttp reset proxy then restart system
ErrImagePull in pods	Check imagePullPolicy and image name; use Never for local images
Service URL not accessible	Run minikube service <service-name> — port changes each restart
Pods stuck in Pending	Check resources; Minikube may not have enough CPU/RAM
kubectl not found	Ensure Minikube installed kubectl — or install separately
Dashboard not opening	Run minikube dashboard and keep terminal open

8 Production Considerations

Aspect	Local (Minikube)	Production (EKS/AKS/GKE)
Nodes	1 (control plane only)	Multiple worker nodes
Control Plane	Local Docker	Cloud-managed
Image Source	minikube image load	ECR/Docker Hub
Monitoring	Minikube dashboard	Prometheus + Grafana
Ingress	minikube service	Ingress controller + LoadBalancer

Aspect	Local (Minikube)	Production (EKS/AKS/GKE)
Storage	Local	Persistent Volumes (EBS, Azure Disk)
Secrets	<code>kubectl create secret</code>	AWS Secrets Manager, Vault
Scaling	Manual replicas	Horizontal Pod Autoscaler (HPA)

What's Next

The instructor plans:

1. **Prometheus + Grafana** (monitoring & alerting)
2. **Seldom** (likely a typo for **Seldon** — model serving on Kubernetes)
3. More **MLOps projects** (CV, interview prep)

Call to action: Comment on the video with topics you want covered next.

Final Thoughts

This two-part Kubernetes series gives you:

- **Solid theory** (Part 1): distributed computing, microservices, K8s internals.
- **Practical skills** (Part 2): Minikube, deployments, services, load balancing.

With this knowledge, you can:

- Confidently discuss Kubernetes in MLOps interviews.
- Deploy containerized ML apps to any Kubernetes cluster.
- Integrate Kubernetes into CI/CD pipelines.
- Understand managed K8s services (EKS/AKS/GKE).

Instructor's words: "This much knowledge is **more than enough** to work as an MLOps professional on the concept of Kubernetes. If you join a big organization already using K8s, you can easily pick up. If you join a startup exploring K8s, you can start from scratch with this knowledge."

End of Kubernetes Part 2 — Hands-On Notes.



ओके ठीक है फिर शुरू करते ...
TXT 166.83KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Monitoring & Alerting — Part 1 (Prometheus & Grafana Fundamentals)

Series: End-to-End MLOps Playlist

Prerequisite: Docker, Kubernetes, Git, basic Python/Flask, CI/CD concepts

Purpose: Build a strong foundation in monitoring, alerting, and observability for MLOps

Why This Topic Matters

When you deploy ML microservices (data ingestion, preprocessing, model training, serving, etc.), **things will go wrong**:

- A service goes down.
- A service becomes slow (high latency).
- A service returns wrong data.
- Traffic spikes overwhelm the system.

Without **monitoring and alerting**, you won't know until users complain — and by then, you've lost business.

Real-world example: If Uber's "connect driver to user" microservice slows down, the user closes the app and switches to Ola. Uber loses revenue.

This two-part series teaches:

Part

Focus

Part 1 (this video)

Fundamentals + local hands-on with Prometheus & Grafana

Part	Focus
Part 2 (next video)	Deploy on Minikube/Kubernetes, connect real ML app, visualize metrics

1 Monitoring vs. Observability — The Core Distinction

Monitoring

Monitoring = Checking and keeping a check on things.

You monitor:

- **Is the service ON?** (Is it accessible to users?)
- **Is it functioning as expected?** (Does it connect to the DB? Does it return the right data?)
- **Is it performing well?** (Latency within expected range?)

Example:

- Expected latency: 60–80 ms.
- You monitor: Is latency within this range?

Observability

Observability = Understanding the *why* behind patterns observed during monitoring.

You observe:

- Patterns over time (daily/weekly trends).
- Increasing latency over weeks.
- Repeated alerts during peak hours.

Example:

- You notice latency has been increasing over 7 days.
- You investigate: Is traffic growing? Is the app inefficient? Do we need scaling?

The Relationship

text

Monitoring → Collect data, check thresholds

Observability → Understand patterns, take action

Interview Tip: This question may appear in **Ops-heavy MLOps roles** — not typically in Data Science/ML-heavy roles.

2 What Data Do We Monitor? — Telemetry

Telemetry = Data collected for monitoring.

Characteristics of Telemetry Data

- **X-axis:** Timestamp (time)
- **Y-axis:** Metric value (latency, CPU usage, request count, etc.)
- **One data point per timestamp.**

Real-World Examples

- **Grafana dashboard:** Lines showing latency over time.
- **Stock price charts** (HDFC, etc.): Price over time.

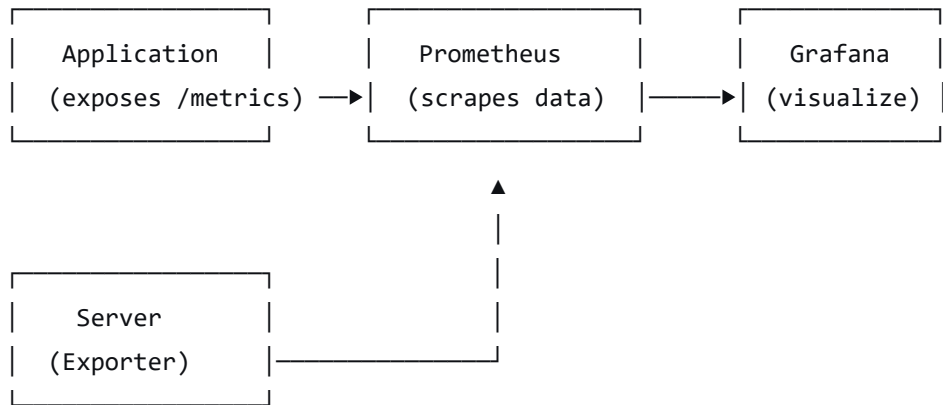
Both are **telemetry data**.

What We Monitor in MLOps

Type	Examples
Application metrics	Latency, request count, error rate, response time
System metrics	CPU, memory, disk, network
Third-party services	MSSQL, CloudWatch, MongoDB (source code not available)

3 Prometheus Architecture — How It Works

text



Components

Component	Role
Application	Exposes metrics via an API endpoint (e.g., /metrics)
Prometheus	Scrapes metrics at regular intervals (default: 15s)
Grafana	Visualizes metrics as dashboards
Exporters	Bridge between servers/third-party services and Prometheus

Two Data Collection Methods

A. Scraping (Most Common)

- Prometheus is given a URL (the app's /metrics endpoint or an exporter's endpoint).
- Prometheus **pulls** data from that URL at regular intervals.
- **Default scrape interval:** 15 seconds.

B. Push Method

- Used for **short-lived jobs** or services that can't be scraped.
- A **middleman** (e.g., **Pushgateway**) receives pushed data.
- Prometheus scrapes from the Pushgateway.

For most MLOps use cases, scraping is the default method.

What is an Exporter?

Exporter = A service that collects metrics from a system and exposes them in a Prometheus-compatible format.

Examples:

- **Node Exporter:** Linux system metrics (CPU, memory, disk).
- **Windows Exporter (WMI):** Windows system metrics (external, but supported).
- **MSSQL Exporter:** SQL Server metrics.

Hands-On: Running Prometheus Locally

Step 1 — Download Prometheus

1. Go to Google → search `prometheus download`.
2. Visit the official site.
3. Select **Windows** (or your OS).
4. Download the ZIP file (~100 MB).

Step 2 — Extract and Run

1. Extract the ZIP.
2. Open PowerShell in the extracted folder.
3. Run:

```
bash
```

```
.\prometheus.exe
```

4. Open `http://localhost:9090` in your browser.

Default port: 9090.

Step 3 — Explore the Prometheus UI

Tab	Purpose
Query	Enter PromQL queries
Graph	Visualize query results
Status → Targets	See what's being monitored
Status → Configuration	See the scrape config

Step 4 — Default Behavior

Prometheus monitors itself by default. Go to **Status → Targets** — you'll see one target: `localhost:9090/metrics` .

Step 5 — Query Prometheus's Own Metrics

In the Query tab, enter:

```
text

go_gc_duration_seconds
```

Click **Execute** → see table results. Click **Graph** → visualize.

Metrics are zero-initialized, so some graphs look flat. Try metrics with non-zero values like `prometheus_http_requests_total` .

Step 6 — Understanding Metrics and Labels

A metric like `prometheus_http_requests_total` has:

- **Metric name:** `prometheus_http_requests_total`
- **Labels:**

```
{code="200", handler="/api/v1/query", instance="localhost:9090", job="prometheus"}
```

Filter by label:

```
text
```

```
prometheus_http_requests_total{handler="/api/v1/query"}
```

This returns only the records matching that label.

MLOps Example: A `data_ingestion_latency` metric might have labels `{source="MSSQL", dataset="train"}`. You can filter for specific sources/datasets.

Step 7 — Time Ranges in PromQL

text

```
metric_name[1m]    → last 1 minute  
metric_name[5m]    → last 5 minutes  
metric_name[1h]    → last 1 hour
```

Why 15 seconds matters: Prometheus scrapes every 15s, so `[1m]` returns ~4 data points.

Step 8 — Arithmetic Operations in PromQL

text

```
metric_name + 1000  
metric_name / 2
```

PromQL supports arithmetic and many functions (abs, ceil, clamp, etc.).

Don't try to learn all of PromQL now. Learn it **on-demand** when you have a requirement. It's a vast topic.

5 Hands-On: Windows Exporter

Step 1 — Download Windows Exporter

1. Search `windows exporter download`.
2. Visit the GitHub releases page.
3. Download the `.exe` (e.g., `windows_exporter-0.29.2-amd64.exe`).

4. Run the installer.

Step 2 — Verify

- Open `http://localhost:9182/metrics` in your browser.
- You'll see a large list of Windows metrics (CPU, memory, disk, etc.).

Default port: 9182.

Step 3 — Add Windows Exporter to Prometheus

Prometheus needs to know about this new target. Edit `prometheus.yml` :

```
yaml

scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "my_windows_machine"
    static_configs:
      - targets: ["localhost:9182"]
```

Restart Prometheus (Ctrl+C in PowerShell, then re-run `.\prometheus.exe`).

Step 4 — Verify in Prometheus

- Go to **Status** → **Targets**.
- You should see **two targets**: `prometheus` and `my_windows_machine` .

Step 5 — Query Windows Metrics

In the Query tab:

```
text

windows_cpu_time_total
```

or

text

```
windows_cpu_time_total{mode="privileged"}
```

Click **Execute** → see results.

Now Prometheus is monitoring your Windows machine's performance.

6 Hands-On: Grafana

Step 1 — Download Grafana

1. Search grafana download .
2. Select Windows.
3. Download and install.

Default port: 3000.

Step 2 — First Login

- Open `http://localhost:3000` .
- Default username/password: `admin / admin` .
- Change password on first login.

Step 3 — Connect Prometheus as a Data Source

1. Go to **Connections** → **Data Sources**.
2. Click **Add data source** → **Prometheus**.
3. Connection URL: `http://localhost:9090` .
4. Click **Save & Test** → should show "Data source is working."

Step 4 — Create a Dashboard

1. Go to **Dashboards** → **New** → **New Dashboard**.
2. Click **Add visualization**.
3. Select **Prometheus** as data source.

4. Enter a PromQL query (e.g., `windows_cpu_time_total`).
5. Use **Builder** (dropdowns) or **Code** mode.
6. Add labels/filters via the **Label filters** section.
7. Set a **Title** (e.g., `First Test Dashboard`).
8. Click **Save** → name the dashboard (e.g., `Grafana Tutorial`) → **Save**.

Step 5 — Add More Panels

- Click **Add visualization** again.
- Add different metrics.
- Arrange and resize panels as needed.
- Save the dashboard.

Step 6 — Explore Panel Options

- **Visualization type:** Time series, gauge, bar chart, table, etc.
- **Legend:** Rename series, show/hide, etc.
- **Axis:** Set units, ranges, etc.
- **Thresholds:** Highlight values above/below a threshold.
- **Alert:** Create alert rules.

Data sources beyond Prometheus: Grafana supports MSSQL, MySQL, PostgreSQL, Elasticsearch, and many more.

7 Hands-On: Alerting in Grafana

Step 1 — Create an Alert Rule

1. Open a dashboard panel.
2. Click **Edit** → go to **Alert** tab.
3. Click **Create alert rule from this panel** (or **New alert rule**).

Step 2 — Configure the Alert

Setting	Purpose
Query	The metric to monitor (e.g., <code>windows_cpu_time_total</code>)
Reduce	Aggregation function (<code>last</code> , <code>mean</code> , <code>min</code> , <code>max</code> , etc.)
Threshold	Value that triggers the alert (e.g., <code>> 9000</code>)
Evaluation behavior	How long the threshold must be breached before firing
Contact point	Where to send notifications (email, Slack, Teams, Jira, etc.)
Notification message	Custom message shown in the alert

Example:

- Query: `windows_cpu_time_total`
- Reduce: `last`
- Threshold: `IS ABOVE 9000`
- Evaluation: `pending for 1m` (or `none` for immediate)

Step 3 — Notification Channels

- **Email:** Requires SMTP configuration in `grafana.ini` .
- **Slack/Teams:** Requires webhook URL.
- **Jira/Confluence:** Requires API token.

Instructor's tip: Setting up email alerts locally is complex. In a company, this is pre-configured. Focus on understanding the concept — the setup burden is on your employer's platform team.

Step 4 — Test the Alert

- Set a low threshold (e.g., `> 5000`).
- Watch the alert fire within a few minutes.
- View it in **Alerting** → **Alert Rules**.

Step 5 — Configure Email Alerts (Optional)

Edit grafana.ini (usually at C:\Program Files\GrafanaLabs\grafana\conf\grafana.ini):

```
ini

[smtp]
enabled = true
host = smtp.gmail.com:587
user = your-email@gmail.com
password = your-app-password
from_address = your-email@gmail.com
from_name = Grafana
```

Restart Grafana. Then create a contact point with the email.

Warning: Use an app password, not your main password.

8 PromQL — A Brief Introduction

PromQL is Prometheus's query language.

Common Operations

Operation	Example
Instant query	metric_name
Range query	metric_name[5m]
Filter by label	metric_name{label="value"}
Arithmetic	metric_name + 1000
Functions	abs(metric_name) , ceil(metric_name)

Common Functions

Function	Purpose
rate()	Per-second rate of a counter
irate()	Instantaneous rate
increase()	Total increase over a time range
avg_over_time()	Average over a time range
sum()	Sum of series
avg()	Average of series
max() , min()	Max/min

Don't try to master PromQL now. Learn on-demand when you have a requirement.

9 Key Takeaways

1. **Monitoring** = checking; **Observability** = understanding patterns.
2. **Telemetry** = time-series data collected for monitoring.
3. **Prometheus** = scrapes metrics; **Grafana** = visualizes them.
4. **Scraping** = pull model; **Pushgateway** = push model for short-lived jobs.
5. **Exporters** = bridge between systems and Prometheus.
6. **Default scrape interval**: 15 seconds.
7. **PromQL** = Prometheus's query language — learn on-demand.
8. **Alerts** = trigger notifications when thresholds are breached.
9. **Grafana** = supports multiple data sources, not just Prometheus.
10. **In production**, Prometheus + Grafana run on dedicated servers (24/7).

10 Command & Config Cheat Sheet

Prometheus

```
bash
```

```
# Run Prometheus (Windows)
.\prometheus.exe
```

```
# Default port
http://localhost:9090
```

prometheus.yml :

```
yaml
```

```
scrape_configs:
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]

  - job_name: "my_windows_machine"
    static_configs:
      - targets: ["localhost:9182"]
```

Windows Exporter

```
bash
```

```
# Run installer
windows_exporter-0.29.2-amd64.exe
```

```
# Default port
http://localhost:9182/metrics
```

Grafana

```
bash
```

```
# Default port
http://localhost:3000
```

```
# Default login
admin / admin
```

1 1 Common Issues & Fixes

Issue	Fix
Prometheus not starting	Check port 9090 is free
Grafana "Data source not working"	Verify Prometheus URL is correct
No data in Grafana	Check Prometheus has scraped the target
Alert not firing	Check threshold, evaluation period, contact point
Email alerts not working	Check <code>grafana.ini</code> SMTP settings
Metrics missing	Verify exporter is running and target is added

1 2 What's Next — Part 2 Preview

In Part 2, we will:

1. Deploy an ML app on **Minikube/Kubernetes**.
2. Expose its metrics via a custom `/metrics` endpoint.
3. Connect Prometheus to scrape those metrics.
4. Build Grafana dashboards for the ML app.
5. Set up alerts on ML-specific metrics (latency, error rate, request count).

Prerequisite: Docker + Kubernetes videos (available in the same playlist).

1 3 Interview Prep — Likely Questions

1. What is monitoring? How is it different from observability?
2. What is telemetry? Give examples.
3. Explain Prometheus architecture.
4. What is scraping? When would you use push instead?

5. What is an exporter? Name a few.
6. What is PromQL? Give an example query.
7. How does Grafana connect to Prometheus?
8. How do you set up an alert in Grafana?
9. What's the default scrape interval in Prometheus?
10. How would you monitor a Kubernetes cluster with Prometheus?

Final Notes

- **Don't over-learn PromQL now.** Learn on-demand.
- **Local setup is for learning.** Production runs on dedicated servers.
- **Email alerting is complex locally.** In a company, it's pre-configured.
- **The goal:** Build a working understanding so you can adapt to any monitoring setup.
- **Next video:** Apply everything to a Kubernetes-deployed ML app.

Instructor's words: "I'm targeting to cover the entire MLOps series in 50–55 hours. If you expect 20 hours per topic, we'd never finish. My goal is to give you the foundation so you can take off anywhere."

End of Monitoring & Alerting — Part 1 Notes.



ओके सो वेलकम बैक टू द एम...
TXT 120.84KB

make notes out of this transcript for me , explain all these topics properly with deep concept and code explanation

Monitoring & Alerting — Part 2 (Prometheus + Grafana on Kubernetes/Minikube)

Series: End-to-End MLOps Playlist

Prerequisite: Docker, Kubernetes (2-part series), Prometheus & Grafana Part 1, Helm basics

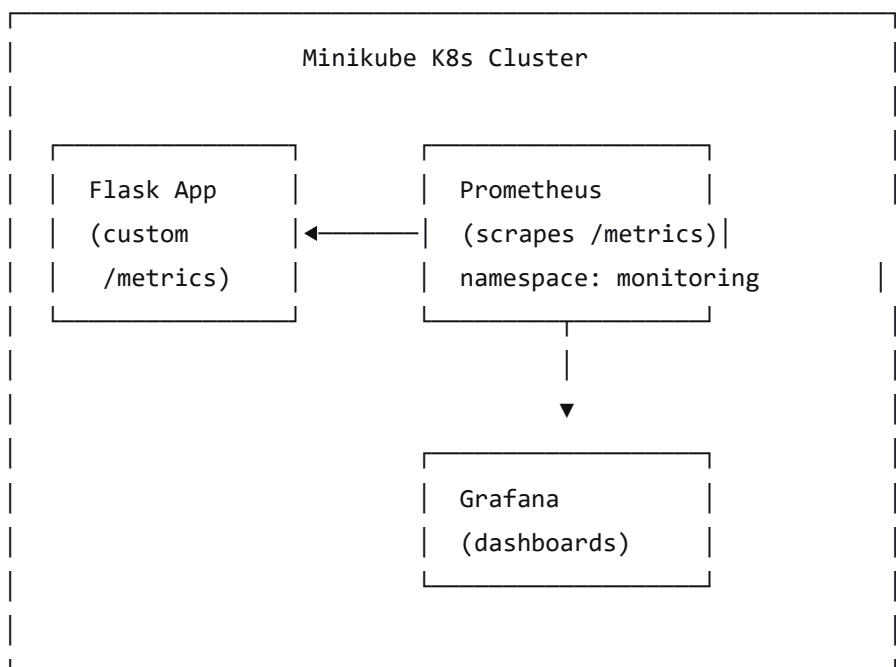
Purpose: Deploy a real monitoring stack (Prometheus + Grafana) on a Minikube Kubernetes cluster and monitor a custom Flask app exposing ML-like metrics

Big Picture — What We're Building Today

In Part 1, we learned the **concepts** of Prometheus, Grafana, scraping, exporters, and telemetry — and ran them **locally on our laptops**.

In Part 2, we **mimic production** by deploying everything on a **Minikube Kubernetes cluster**:

text



Key learning outcomes:

1. Deploy a custom Flask app with Prometheus metrics to Kubernetes.
2. Install Prometheus and Grafana in a K8s cluster using **Helm**.
3. Configure Prometheus to scrape the app's `/metrics` endpoint.
4. Connect Grafana to Prometheus and build dashboards.

Why a "Fake" ML App (Not a Real ML App)?

The instructor uses a simple Flask app that **mimics ML metrics behavior** instead of a real ML app.

Reasons:

Reason	Explanation
Local machine limits	Running a full ML app + K8s cluster + Prometheus + Grafana would crash a typical laptop
Video length	Building a real ML app took 5 videos in the playlist — too long to redo here
Focus	The goal is learning K8s monitoring , not building ML apps

If you have a powerful machine (32GB+ RAM): You can replace the Flask app with your real ML app — the monitoring setup is identical.

The Flask App — app.py

```
python

from flask import Flask, Response
import random
import time

app = Flask(__name__)

# Global counters
total_requests = 0

@app.route("/")
def home():
    return "Welcome to the Prometheus Metrics Demo App"

@app.route("/metrics")
def metrics():
    global total_requests
    total_requests += 1

    # Simulate ML-like metrics with random values
    latency = random.uniform(0.1, 2.0)
    success_rate = random.uniform(0.85, 1.0)
```

```
metrics_output = f"""# HELP total_api_requests_total Total API requests
# TYPE total_api_requests_total counter
total_api_requests_total {total_requests}

# HELP request_processing_latency_seconds Request processing latency
# TYPE request_processing_latency_seconds gauge
request_processing_latency_seconds {latency}

# HELP model_prediction_success_rate Model prediction success rate
# TYPE model_prediction_success_rate gauge
model_prediction_success_rate {success_rate}
"""

return Response(metrics_output, mimetype="text/plain")

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

The Three Metrics

Metric	Type	Meaning
total_api_requests_total	Counter	Total number of API requests
request_processing_latency_seconds	Gauge	Current request latency
model_prediction_success_rate	Gauge	Model prediction success rate

Prometheus Metrics Format

Every metric follows this format:

```
text

# HELP <metric_name> <description>
# TYPE <metric_name> <type>
<metric_name> <value>
```

- # HELP = description
- # TYPE = counter/gauge/histogram/summary
- <metric_name> <value> = the actual data point

This is the exact format used by Windows Exporter, Prometheus itself, and every exporter.

Test Locally

```
bash
```

```
python app.py
```

```
# Open http://localhost:5000 → "Welcome..."
```

```
# Open http://localhost:5000/metrics → the three metrics
```

2 Dockerize the Flask App

Dockerfile

```
dockerfile
```

```
FROM python:3.9-slim
```

```
WORKDIR /app
```

```
COPY . /app
```

```
RUN pip install flask
```

```
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Build the Image

```
bash
```

```
docker build -t flask-metrics-app:latest .
```

Verify

```
bash
```

`docker` images

Should show flask-metrics-app:latest

Image size: ~130 MB for this lightweight app.

3 Kubernetes Manifest — flask-app.yaml

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-metrics-app
  labels:
    app: flask-metrics-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-metrics-app
  template:
    metadata:
      labels:
        app: flask-metrics-app
    spec:
      containers:
        - name: flask-metrics-app
          image: flask-metrics-app:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: flask-metrics-app-service
spec:
  selector:
    app: flask-metrics-app
  ports:
    - protocol: TCP
      port: 5000
```

```
targetPort: 5000
type: NodePort
```

Key points:

- `imagePullPolicy: Never` — use the locally loaded image.
- `NodePort` — exposes the app for local access.

4 Start the Minikube Cluster

```
bash
```

```
# Delete any existing cluster
minikube delete --all
```

```
# Start fresh
minikube start
```

Important: All subsequent `kubectl` commands must be run from **inside the project directory** (or use `-f` with the correct path).

5 Load the Docker Image into Minikube

Minikube runs in its own Docker environment — it can't see your local images by default.

```
bash
```

```
minikube image load flask-metrics-app:latest
```

Verify:

```
bash
```

```
minikube image list
# Should now include flask-metrics-app:latest
```

6 Deploy the Flask App

```
bash
```

```
kubectl apply -f flask-app.yaml
```

Verify:

```
bash
```

```
kubectl get pods
```

```
kubectl get services
```

Access the App

```
bash
```

```
minikube service flask-metrics-app-service --url
```

This prints a URL like `http://127.0.0.1:56008` .

Test:

- Open `http://127.0.0.1:56008/` → "Welcome..."
- Open `http://127.0.0.1:56008/metrics` → the three metrics

Note: The port changes every time you restart the Minikube service.

7 Introduction to Helm

What is Helm?

Helm = The package manager for Kubernetes.

Analogy: Just like `pip` is for Python and `npm` is for Node.js, **Helm is for Kubernetes.**

Tool	Ecosystem
<code>pip</code>	Python packages

Tool	Ecosystem
npm	Node.js packages
apt / yum	Linux packages
Helm	Kubernetes applications

Why Helm?

- Deploying Prometheus manually requires dozens of YAML files (Deployments, Services, ConfigMaps, RBAC, etc.).
- Helm **packages** all of these into a **chart** (like a bundle).
- One command installs everything.

Helm Terminology

Term	Meaning
Chart	A package of K8s manifests
Release	An instance of a chart installed in a cluster
Repository	A collection of charts

8 Install Helm on Windows

Option 1 — winget (easiest)

```
powershell
```

```
winget install Helm.Helm
```

Option 2 — Chocolatey

```
powershell
```

```
choco install kubernetes-helm
```

Option 3 — Scoop

```
powershell
```

```
scoop install helm
```

Verify:

```
bash
```

```
helm version
```

Install Prometheus with Helm

Step 1 — Add the Helm Repository

```
bash
```

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
```

Step 2 — Update Repositories

```
bash
```

```
helm repo update
```

Step 3 — Search for Prometheus Charts

```
bash
```

```
helm search repo prometheus
```

You'll see many charts (prometheus , kube-prometheus-stack , prometheus-node-exporter , etc.).

Step 4 — Install Prometheus

```
bash
```

```
helm install prometheus prometheus-community/prometheus \
  --namespace monitoring \
  --create-namespace
```

What this does:

- Creates the `monitoring namespace` (virtual cluster).
- Installs Prometheus in it.
- Also installs `node-exporter`, `kube-state-metrics`, `alertmanager`, and `pushgateway`.

Why a Separate Namespace?

Isolation: If your app server crashes, monitoring keeps running. If monitoring crashes, your app keeps running. **Separate namespaces = separate concerns.**

Real-world analogy: In production, monitoring runs on a **dedicated server** (not on the same server as your app).

Verify Installation

```
bash
```

```
kubectl get pods -n monitoring
kubectl get services -n monitoring
```

Access Prometheus UI

Option 1 — Port Forwarding (Bash):

```
bash
```

```
export POD_NAME=$(kubectl get pods --namespace monitoring -l "app.kubernetes.io/name=prometheus,app.kubernetes.io/instance=prometheus" -o jsonpath="{.items[0].metadata.name}")
kubectl --namespace monitoring port-forward $POD_NAME 9090
```

Option 2 — Port Forwarding (PowerShell/Any OS):

```
bash
```

```
kubectl port-forward -n monitoring svc/prometheus-server 9090:80
```

Open `http://localhost:9090` .

Why :80 ? Prometheus's K8s Service listens on port **80** internally, but we forward it to **9090** locally.

10 Configure Prometheus to Scrape the Flask App

Step 1 — Get the Flask App's Cluster IP and Port

```
bash
```

```
kubectl get services  
# Find flask-metrics-app-service  
# Note its CLUSTER-IP and PORT (e.g., 10.107.61.186:5000)
```

Step 2 — Edit Prometheus's ConfigMap

```
bash
```

```
kubectl edit configmap prometheus-server -n monitoring
```

This opens the ConfigMap in your default text editor (Notepad on Windows).

 **Warning:** Editing YAML in a text editor is risky. One wrong space = error. Be careful with indentation.

Step 3 — Add the Scrape Config

Find the `scrape_configs:` section. It currently has `prometheus` (scraping itself). Add a new job:

```
yaml
```

```
scrape_configs:  
  - job_name: prometheus
```

```
static_configs:
  - targets: ['localhost:9090']

- job_name: flask-metrics-app
  static_configs:
    - targets: ['10.107.61.186:5000']
```

Indentation rules:

- scrape_configs: at column 0.
- - job_name: indented 2 spaces.
- static_configs: indented 4 spaces.
- - targets: indented 6 spaces.

Save and close the editor.

Step 4 — Restart Prometheus

```
bash
```

```
kubectl rollout restart deployment prometheus-server -n monitoring
```

Verify:

```
bash
```

```
kubectl get pods -n monitoring
# Prometheus pod should restart
```

Step 5 — Re-do Port Forwarding

```
bash
```

```
kubectl port-forward -n monitoring svc/prometheus-server 9090:80
```

Step 6 — Verify in Prometheus UI

1. Open `http://localhost:9090` .
2. Go to **Status** → **Targets**.

3. You should see **two targets**: `prometheus` and `flask-metrics-app` .

Step 7 — Query the Custom Metrics

In the Query tab:

```
text
```

```
total_api_requests_total
```

Click **Execute** → see the counter value.

Refresh the app's `/metrics` **endpoint** several times → re-execute → the counter increases.

Why doesn't it increase immediately? Prometheus scrapes every **15 seconds** by default.

Metrics with Labels

Query:

```
text
```

```
total_api_requests_total{instance="10.107.61.186:5000"}
```

- `instance` = the target's address (identifies which deployment).
- `job` = the scrape job name.

Use case: If you have multiple apps exposing the same metric, use labels to filter for a specific app.

1 1 Install Grafana with Helm

Step 1 — Add the Grafana Helm Repo

```
bash
```

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
```

Step 2 — Install Grafana

bash

```
helm install grafana grafana/grafana \
  --namespace monitoring \
  --create-namespace
```

Step 3 — Verify

bash

```
kubectl get pods -n monitoring
kubectl get services -n monitoring
# Look for grafana service, port 80
```

Step 4 — Get the Grafana Admin Password

Bash:

bash

```
kubectl get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}" |
base64 --decode ; echo
```

PowerShell:

powershell

```
# Get the base64-encoded password
kubectl get secret --namespace monitoring grafana -o jsonpath="{.data.admin-password}"

# Decode it (replace <BASE64_STRING> with the output above)
[System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String("<BASE64_STRING
>"))
```

Step 5 — Port Forward Grafana

```
bash
```

```
kubectl port-forward -n monitoring svc/grafana 3000:80
```

Open `http://localhost:3000` .

Login:

- Username: `admin`
- Password: the decoded password (ends with `gh6` in the video)

1 2 Connect Grafana to Prometheus

1. In Grafana: **Connections** → **Data Sources** → **Add data source**.
2. Select **Prometheus**.
3. **URL:** `http://prometheus-server.monitoring.svc.cluster.local:80`
Or simply: `http://prometheus-server:80` (if Grafana is in the same namespace).
Or: `http://prometheus-server.monitoring.svc.cluster.local:80`
4. **HTTP Method:** `GET`
5. Click **Save & Test** → should show "Data source is working."

Why not `localhost:9090` ? Because Grafana runs **inside the cluster**, not on your laptop. It needs the **cluster-internal** DNS name of the Prometheus service.

1 3 Build a Dashboard

Step 1 — Create Dashboard

1. Go to **Dashboards** → **New** → **New Dashboard**.
2. Click **Add visualization**.
3. Select **Prometheus** as the data source.

Step 2 — Add the First Panel

1. Enter the query: `total_api_requests_total`
2. Click **Run query**.
3. Observe the metric.
4. Set the panel title: `First ML Dashboard`.
5. Click **Save dashboard** → name it `My First ML Dashboard`.

Step 3 — Add More Panels

Click **Add visualization** again.

Try:

- `request_processing_latency_seconds`
- `model_prediction_success_rate`

Save after each.

Step 4 — Arrange Panels

- Drag and resize panels.
- Rename legends for clarity.

Step 5 — Use Labels for Filtering

Query:

`text`

`total_api_requests_total{instance="10.107.61.186:5000"}`

This shows only the data from a specific deployment.

Use case: If you have 10 ML apps in the cluster, each with the same metrics, filter by `instance` to see one app's data.

Deep Concept Recap

Why Namespaces?

- **Isolation:** Monitoring and apps don't interfere.
- **Resource limits:** Set quotas per namespace.
- **Access control:** RBAC can be scoped per namespace.

Why Helm?

- **Simplifies deployment:** One command instead of dozens of YAMLs.
- **Versioning:** Charts are versioned.
- **Reproducibility:** Same chart = same deployment everywhere.

Why Labels?

- **Filtering:** Query only specific instances.
- **Aggregation:** Group metrics by label.
- **Multi-tenancy:** Same metric name for different apps.

Why 15 Seconds?

- **Default scrape interval** balances freshness and load.
- Shorter = more up-to-date but more load.
- Longer = less load but stale data.

1 5 Command Cheat Sheet

bash

Minikube

minikube start

minikube delete --all

minikube image load <image>:<tag>

minikube image list

minikube service <service-name> --url

HeLm

helm repo add <name> <url>

helm repo update

```
helm search repo <chart>
helm install <release> <chart> -n <namespace> --create-namespace
helm list -n <namespace>
helm uninstall <release> -n <namespace>

# kubectl
kubectl apply -f <file>.yaml
kubectl get pods -n <namespace>
kubectl get services -n <namespace>
kubectl get svc -n <namespace>
kubectl edit configmap <name> -n <namespace>
kubectl rollout restart deployment <name> -n <namespace>
kubectl port-forward -n <namespace> svc/<service> <local>:<remote>
kubectl get secret <name> -n <namespace> -o jsonpath="{.data.<key>}" | base64 --decode
```

1 6 Common Issues & Fixes

Issue	Fix
ErrImagePull	Check imagePullPolicy: Never and minikube image load
Prometheus pod not restarting	Check ConfigMap YAML indentation
Grafana can't connect to Prometheus	Use cluster-internal DNS (prometheus-server:80)
Metrics not increasing	Wait 15 seconds (scrape interval)
Port changes after restart	Re-run minikube service --url
ConfigMap edit broke YAML	Restart the pod; revert if needed

1 7 Key Takeaways

1. **Monitoring on K8s mimics production** — separate namespace, Helm-managed, remote scraping.
2. **Helm = pip for Kubernetes** — packages K8s manifests into charts.
3. **Namespace** monitoring isolates monitoring from apps.

4. **Prometheus scrapes every 15 seconds** by default.
5. **Labels** (`instance` , `job`) identify specific deployments.
6. **Grafana uses cluster-internal DNS** to reach Prometheus.
7. **Custom metrics** are exposed via `/metrics` in Prometheus format.
8. **Port forwarding** lets you access cluster services locally.
9. **One command via Helm** installs Prometheus, Alertmanager, Node Exporter, Kube State Metrics, Pushgateway.
10. **The same process** applies to production (EKS, AKS, GKE, on-prem).

1 8 What's Next

- **Seldon Core** — model serving on Kubernetes (next video).
- **Alertmanager** — deep dive into alert routing.
- **Distributed tracing** — Jaeger, OpenTelemetry.
- **Log aggregation** — EFK stack (Elasticsearch, Fluentd, Kibana) or Loki.

Instructor's words: "I'm planning Seldon Deploy next — a very popular topic in the MLOps umbrella. Tell me what else you'd like to see."

1 9 Interview Prep — Likely Questions

1. How do you deploy Prometheus on Kubernetes?
2. What is Helm? Why use it?
3. What is a namespace? Why use one for monitoring?
4. How do you configure Prometheus to scrape a custom app?
5. What is the default scrape interval?
6. How does Grafana connect to Prometheus in K8s?
7. What are labels? How are they used in PromQL?
8. How do you expose custom metrics from a Flask/ML app?
9. What is the difference between `counter` and `gauge` ?
10. How would you monitor multiple ML services in one cluster?

End of Monitoring & Alerting — Part 2 Notes.

give me interview question with answer of all the things covered in this chat topicwise

MLOps Interview Questions & Answers — Topic-wise

Based on all the notes covered in this chat (Parts 1–7 + Kubernetes + Prometheus/Grafana)

TOPIC 1: Python Project Structure & Best Practices

Q1. Why do we need a `__init__.py` file in every folder?

Answer: `__init__.py` tells Python that a folder is a **package** and can be imported. Without it, Python won't recognize the folder as an importable module. In our project, we added local packages to the virtual environment, so every folder (`constants/` , `components/` , `configuration/` , etc.) needs `__init__.py` for imports to work.

Q2. Why separate constants into a separate file?

Answer: To avoid **code redundancy** and improve **maintainability**. If the same value (e.g., database name, file path) is used in 5 places and needs to change, you change it once in the constants file instead of hunting through the codebase. This is a production-grade practice.

Q3. What is the difference between a config entity and an artifact entity?

Answer:

- **Config Entity:** A dataclass holding **input configuration** (paths, parameters, thresholds) needed by a component.
- **Artifact Entity:** A dataclass holding **outputs** produced by a component (e.g., file paths, metrics), which are passed to the next component.

Q4. Why use dataclasses for config and artifact entities?

Answer: They're **clean, typed, and self-documenting**. They provide automatic `__init__`, `__repr__`, and type hints, making the code readable and reducing boilerplate.

TOPIC 2: Logging & Exception Handling

Q5. Why create a custom exception module?

Answer: Python's default exceptions don't give **file name and line number** where the error occurred. Our `CustomException` captures the traceback and adds file/line info, making **debugging across many files** much easier. It's essential in multi-file production projects.

Q6. How does logging help in a distributed system?

Answer: Logging provides a **trace of execution** across components. In distributed systems (microservices, pipelines), you can trace which service failed, at what step, and with what data. It's critical for **observability** and **debugging**.

Q7. What's the difference between logging and print?

Answer:

- `print()` → output to console only, no levels, no timestamps, no file storage.
- `logging` → supports **levels** (INFO, DEBUG, WARNING, ERROR, CRITICAL), **timestamps**, **file handlers**, and **custom formats**. It's production-grade.

TOPIC 3: MongoDB & Data Ingestion

Q8. Why use environment variables for MongoDB URL instead of hardcoding?

Answer:

- **Security:** Never commit credentials to Git.

- **Flexibility:** Different environments (dev/staging/prod) can use different URLs without code changes.
- **Production standard:** Every real job uses this pattern.

Q9. What is the singleton pattern in MongoClient?

Answer: The client is created **only once** at the class level (`client = None`). On subsequent instantiations, the existing client is reused. This avoids creating multiple connections, saving resources and improving performance.

Q10. What is a feature store?

Answer: A **feature store** is a centralized repository where processed features are stored for training and inference. In our project, the raw data from MongoDB is saved as `data.csv` in the `feature_store/` directory. It ensures **consistency** between training and serving.

Q11. Why save the preprocessing object (pickle)?

Answer: To ensure **consistent transformation** during inference. New data must be transformed **exactly the same way** as training data. Loading the saved preprocessor and calling `.transform()` guarantees this consistency.

Q12. What is data versioning? How did we implement it?

Answer: Data versioning tracks different versions of datasets/models. We implemented a **timestamped artifact directory** (`artifact/<timestamp>/`) as a poor-man's versioning workaround. Tools like **DVC** are better, but this approach works for simple projects.

TOPIC 4: Data Validation

Q13. Why do we need a data validation component?

Answer: To **fail early** if the data doesn't match expectations. If columns are missing, data types are wrong, or schema has changed, there's no point in training a model on bad data. It saves compute and prevents garbage models.

Q14. What does the schema YAML contain?

Answer:

- Expected columns
- numerical_columns and categorical_columns
- drop_columns (e.g., _id)
- num_features (for StandardScaler)
- mm_columns (for MinMaxScaler)

Q15. How does Data Validation check data?

Answer:

1. `validate_number_of_columns()` — compares schema's expected count with the dataframe's actual count.
2. `is_column_exist()` — checks each numerical and categorical column from schema exists.
3. Accumulates errors in `validation_error_msg`.
4. Saves report to `report.yaml` with `validation_status` and `message`.

TOPIC 5: Data Transformation

Q16. What is a ColumnTransformer? Why use it?

Answer: `ColumnTransformer` applies different transformers to different column subsets in one step. In our project:

- Numerical columns → `SimpleImputer(median)` + `StandardScaler`
- Categorical columns → `SimpleImputer(most_frequent)` + `OneHotEncoder` + `StandardScaler`

This is cleaner than transforming columns separately.

Q17. What is SMOTEENN? Why use it?

Answer: SMOTEENN = SMOTE (Synthetic Minority Oversampling Technique) + ENN (Edited Nearest Neighbours).

- SMOTE oversamples the minority class.
- ENN removes noisy/ambiguous samples.
- Together they **balance the dataset** and clean noisy samples, improving model performance on imbalanced data.

Q18. Why save transformed data as .npy instead of .csv ?

Answer: .npy is NumPy's **binary format** — much faster to read/write and more memory-efficient for large arrays. .csv is text-based and slower.

Q19. What's the difference between fit_transform and transform ?

Answer:

- `fit_transform()` : Learns parameters (mean, std, etc.) from data **and** applies the transformation. Used on **training data**.
- `transform()` : Applies the **already-learned** transformation. Used on **test data** and **new data** during inference.

Never use `fit_transform()` on test data — that would cause **data leakage**.

TOPIC 6: Model Trainer

Q20. Why use RandomForestClassifier?

Answer: It's an **ensemble of decision trees** that:

- Handles both numerical and categorical data well.
- Is robust to outliers.
- Provides feature importance.
- Generally performs well on tabular data without heavy tuning.

Q21. Why wrap the model + preprocessor in MyModel ?

Answer: In production, when you call `model.predict(new_data)`, the preprocessor must be applied first. The `MyModel` wrapper ensures **both happen together** — no mistakes, consistent transformations.

Q22. What is the expected accuracy check for?

Answer: If the trained model's accuracy is **below the threshold** (e.g., 0.6), the pipeline raises an exception and doesn't save the model. This prevents deploying a **bad model** to production.

Q23. What metrics do we use for classification?

Answer:

- **Accuracy:** Overall correctness.
- **F1 Score:** Harmonic mean of precision and recall — good for imbalanced data.
- **Precision:** True positives / (True positives + False positives).
- **Recall:** True positives / (True positives + False negatives).

TOPIC 7: Model Evaluation & Model Pusher

Q24. Why compare the new model with the previously deployed model?

Answer: Not every new model is better. Deploying a worse model **degrades production performance**. We only push the new model if it's better by a threshold (e.g., 2% improvement in F1 score).

Q25. What is `changed_threshold_score` ?

Answer: The **minimum improvement** required to accept a new model. If $\text{new_f1} - \text{old_f1} < \text{changed_threshold_score}$, the new model is rejected.

Q26. What is the Model Pusher's role?

Answer: To upload the accepted model to S3 (the remote model registry). It only pushes if `is_model_accepted == True`, protecting production from worse models.

Q27. Why S3 for model storage?

Answer:

- **Centralized:** One source of truth for the deployed model.
- **Versioned:** S3 keeps history.
- **Accessible:** Any service can pull from S3.
- **Production-standard:** Mirrors real MLOps workflows (model registry).

TOPIC 8: AWS (IAM, S3, ECR, EC2)

Q28. Why create an IAM user instead of using root?

Answer:

- **Security:** If credentials leak, damage is limited.
- **Access control:** Different users get different permissions.
- **Auditability:** Track who did what.
- **Best practice:** Never use root for daily work.

Q29. What is ECR? How is it different from Docker Hub?

Answer: ECR (Elastic Container Registry) is AWS's Docker image registry. Unlike Docker Hub:

- It's **private by default**.
- Integrates with IAM for access control.
- Works seamlessly with ECS/EKS/EC2.
- Same region as other AWS services.

Q30. What is the difference between EC2 and ECR?

Answer:

- **ECR** = Docker image registry (storage).
- **EC2** = Virtual server (compute).
- ECR stores the image; EC2 pulls it and runs the container.

Q31. What is a Security Group?

Answer: A virtual firewall for EC2 instances. It controls **inbound** and **outbound** traffic rules. We opened port 5000 to make our app publicly accessible.

Q32. Why use environment variables for AWS credentials?

Answer: Same reason as MongoDB URL — **security** and **flexibility**. Never hardcode credentials. Use `os.getenv("AWS_ACCESS_KEY_ID")` etc.



TOPIC 9: FastAPI & Prediction Pipeline

Q33. Why FastAPI over Flask?

Answer:

- **Faster:** Async-first, high performance.
- **Type-safe:** Uses Python type hints.
- **Auto docs:** Swagger UI at `/docs`.
- **Modern:** Great for ML serving.

Q34. What is CORS? Why do we need it?

Answer: **CORS (Cross-Origin Resource Sharing)** is a browser security mechanism. By default, a page from origin A can't make requests to origin B. CORS middleware tells the browser "I allow these origins". Without it, your deployed app might not work when accessed from a different domain/port.

Q35. What is `async / await` ? Why use it?

Answer:

- `async def` declares a **coroutine** that can be paused and resumed.
- `await` pauses the current coroutine until the awaited operation completes, **letting other coroutines run**.
- **Benefit:** While one request is waiting (DB, network, model inference), the server handles other requests. Makes the app **responsive under load**.

Q36. What is the difference between GET and POST routes?

Answer:

- **GET** — retrieves data (e.g., render the form).
- **POST** — submits data (e.g., form submission → prediction).

FastAPI differentiates routes by **HTTP method**, so the same path / can have both GET and POST handlers.

Q37. Why separate the prediction pipeline from the training pipeline?

Answer:

- **Separation of concerns:** Training and inference are different workflows.
- **Reusability:** The prediction pipeline can be used in CLI, batch jobs, or web API.
- **Independence:** Training can be updated without affecting inference.

TOPIC 10: Docker

Q38. What is the difference between a Docker image and a container?

Answer:

- **Image:** Read-only template with app + dependencies.
- **Container:** Running instance of an image.

Q39. Why use `python:3.9-slim` instead of `python:3.9` ?

Answer: `slim` is a **lightweight** base image that excludes unnecessary packages, reducing image size. Smaller image = faster push/pull, lower storage cost.

Q40. What is the purpose of `.dockerignore` ?

Answer: It excludes files/folders from being copied into the image (e.g., `venv/` , `__pycache__/` , `.git/` , `artifacts/`). This keeps the image **small and clean**.

Q41. What does `EXPOSE 5000` do?

Answer: It **documents** that the container listens on port 5000. It doesn't actually publish the port — that's done with `-p 5000:5000` when running.

Q42. What is port mapping?

Answer: `-p host:container` forwards traffic from the host port to the container port. E.g., `-p 5000:5000` means host port 5000 → container port 5000.

TOPIC 11: CI/CD (GitHub Actions)

Q43. What is CI vs CD?

Answer:

- **CI (Continuous Integration):** Automatically build and test code on every push.
- **CD (Continuous Deployment/Delivery):** Automatically deploy the built artifact to production.

Q44. What is a self-hosted runner?

Answer: A machine (e.g., your EC2) that executes GitHub Actions jobs, instead of GitHub's servers. Used so deployment happens **on the target server** without SSH.

Q45. What is `needs:` in a GitHub Actions workflow?

Answer: It defines **job dependencies**. E.g., `needs: continuous-integration` means the CD job waits for CI to succeed before running.

Q46. Why use GitHub Secrets?

Answer: To store sensitive values (API keys, passwords) securely. They're injected at runtime and never exposed in logs or the repo.

Q47. What is the purpose of the `paths` filter in a workflow trigger?

Answer: It runs the workflow **only when relevant files change** (e.g., `src/**`, `Dockerfile`). Avoids unnecessary runs when editing README or other irrelevant files.

TOPIC 12: Kubernetes — Theory

Q48. What is distributed computing?

Answer: A system where **multiple computers/nodes work together** to solve a large problem. Tasks are divided among nodes using **parallel processing** for faster computation. Example: computing log of billions of records across multiple machines.

Q49. What is the difference between monolithic and microservices architecture?

Answer:

- **Monolithic:** All components bundled into one application. Hard to scale partially.
- **Microservices:** Small, independent services communicating via APIs. Each can be scaled, deployed, and updated independently.

Q50. What is a Kubernetes cluster?

Answer: A group of interconnected computers (nodes) working as a single system. It consists of the **Control Plane** (master) and **Worker Nodes**.

Q51. What is the Control Plane? Name its components.

Answer: The **brain** of the cluster. Components:

- **API Server** — interface between users and cluster.

- **etcd** — cluster database (key-value store).
- **Resource Manager / Scheduler** — allocates resources and schedules pods.
- **Controller Manager** — manages controllers (ReplicaSets, Deployments, etc.).

Q52. What is a Pod?

Answer: The **smallest deployable unit** in Kubernetes. A pod wraps **one or more containers**. Usually one container per pod.

Q53. What is a Service in Kubernetes?

Answer: A **stable network endpoint** for a set of pods. Since pods are ephemeral (IPs change), a Service provides a single stable address that routes to currently running pods.

Q54. What is a Namespace?

Answer: A **virtual cluster** within a Kubernetes cluster. Used to organize and isolate resources (e.g., `monitoring`, `default`, `kube-system`).

Q55. What is a ReplicaSet?

Answer: Ensures a **specified number of identical pods** are always running. If one pod dies, ReplicaSet spins up a new one.

Q56. What is the difference between a Deployment and a ReplicaSet?

Answer:

- **ReplicaSet:** Ensures N pods are running.
- **Deployment:** Manages ReplicaSets; supports **rolling updates** and **rollbacks**. Always prefer Deployments.

Q57. What is Kubelet?

Answer: An **agent** on each worker node that ensures containers are running as expected. It reports health/failures back to the Control Plane.

Q58. What is Kube-proxy?

Answer: Manages **network traffic** for pods. Handles pod-to-pod communication, load balancing, and routing.

Q59. What is a Kubemanifest?

Answer: A **YAML file** defining Kubernetes resources (Deployment, Service, etc.). It's the **blueprint** for what you want K8s to do.

Q60. What is the difference between Ingress and Egress?

Answer:

- **Ingress:** Traffic coming **into** the cluster.
- **Egress:** Traffic going **out** of the cluster.

TOPIC 13: Kubernetes — Hands-On

Q61. What is Minikube?

Answer: A tool that runs a **single-node Kubernetes cluster** on your local machine. Used for learning and development, **not production**.

Q62. What is the difference between minikube image load and pushing to a registry?

Answer:

- **minikube image load** — loads a local Docker image directly into the Minikube cluster (dev).
- **Registry push** — pushes to ECR/Docker Hub; cluster pulls from there (production).

Q63. What is imagePullPolicy: Never ?

Answer: Tells Kubernetes to **never pull** from a remote registry — use only the **local image**. Used when we've loaded the image via `minikube image load` .

Q64. How does self-healing work in Kubernetes?

Answer: If a pod fails or is deleted, the **ReplicaSet** (managed by a Deployment) detects the discrepancy and **spins up a new pod** to maintain the desired replica count. This happens in seconds.

Q65. What is port forwarding in Kubernetes?

Answer: `kubectl port-forward` forwards traffic from a **local port** to a **pod or service** inside the cluster. Useful for accessing services during development.

Q66. What is `kubectl` ? Is it different for different K8s environments?

Answer: `kubectl` is the **CLI tool** for interacting with any Kubernetes cluster. Commands are **identical** across Minikube, EKS, AKS, GKE, and on-prem clusters.

Q67. What is the difference between `kubectl get pods` and `kubectl get all` ?

Answer:

- `kubectl get pods` — lists only pods.
- `kubectl get all` — lists all resources (pods, services, deployments, replicaset, etc.).

Q68. How do you scale a deployment?

Answer: Edit the `replicas` field in the deployment YAML and `kubectl apply -f`, or use:

```
bash
```

```
kubectl scale deployment <name> --replicas=5
```

Q69. What is the Minikube dashboard? Is it used in production?

Answer: A web UI for visualizing cluster state (pods, deployments, services). It's a **Minikube-specific learning tool**. In production, **Prometheus + Grafana** are used.

Q70. What is the difference between a Deployment and a Service YAML?

Answer:

- **Deployment YAML:** Defines the pod template, replicas, image, resources.
- **Service YAML:** Defines the stable endpoint, selector, ports, and type (ClusterIP/NodePort/LoadBalancer).

TOPIC 14: Helm

Q71. What is Helm?

Answer: The **package manager for Kubernetes** — like `pip` for Python or `npm` for Node.js. It packages K8s manifests into **charts** and installs them with one command.

Q72. What is a Helm chart?

Answer: A **package** of Kubernetes manifests. Includes templates, values, and metadata.
`helm install` deploys the chart as a **release**.

Q73. What is the difference between a Helm chart and a Helm release?

Answer:

- **Chart:** The package (template).
- **Release:** An instance of the chart installed in a cluster.

Q74. Why use Helm instead of raw `kubectl apply` ?

Answer:

- **Simplifies complex deployments** (Prometheus requires dozens of YAMLs).
- **Versioning** — charts are versioned.
- **Reproducibility** — same chart = same deployment.
- **Parameterization** — values can be overridden per environment.

Q75. How do you install Prometheus with Helm?

Answer:

```
bash

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
helm install prometheus prometheus-community/prometheus \
  --namespace monitoring --create-namespace
```

TOPIC 15: Prometheus & Grafana

Q76. What is monitoring vs observability?

Answer:

- **Monitoring:** Checking and keeping a check on things (is the service on? is it performing well?).
- **Observability:** Understanding **why** behind patterns observed during monitoring (trends, increasing latency, repeated alerts).

Q77. What is telemetry data?

Answer: Data collected for monitoring. Characteristics:

- X-axis: Timestamp
 - Y-axis: Metric value
 - One data point per timestamp.
- Examples: Grafana dashboards, stock price charts.

Q78. What is Prometheus? How does it work?

Answer: An open-source monitoring and alerting toolkit. It:

1. **Scrapes** metrics from targets (apps, exporters) at regular intervals (default: 15s).
2. Stores them as **time-series data**.
3. Allows querying via **PromQL**.

4. Integrates with Grafana for visualization.

Q79. What is scraping vs push method?

Answer:

- **Scraping:** Prometheus **pulls** metrics from a target's URL. Default method.
- **Push:** The target **pushes** metrics to a middleman (e.g., Pushgateway), which Prometheus scrapes. Used for short-lived jobs.

Q80. What is an exporter?

Answer: A service that **collects metrics** from a system and **exposes them** in Prometheus format. Examples: Node Exporter (Linux), Windows Exporter (WMI), MSSQL Exporter.

Q81. What is PromQL?

Answer: Prometheus's **query language**. Used to query and manipulate time-series data.
Example:

text

```
total_api_requests_total{instance="10.107.61.186:5000"}
```

Supports arithmetic, functions (`rate` , `avg` , `sum`), and time ranges (`[5m]`).

Q82. What is a metric vs a label?

Answer:

- **Metric:** The name of the measurement (e.g., `total_api_requests_total`).
- **Label:** Key-value metadata (e.g.,
`{instance="10.107.61.186:5000", job="flask-metrics-app"}`).

Q83. What is the default scrape interval in Prometheus?

Answer: 15 seconds. Can be changed in `prometheus.yml` .

Q84. What is Grafana?

Answer: An open-source **visualization and dashboarding tool**. Connects to data sources (Prometheus, MSSQL, etc.) and displays metrics as graphs, tables, gauges.

Q85. How does Grafana connect to Prometheus in Kubernetes?

Answer: Using the **cluster-internal DNS name**:

text

```
http://prometheus-server.monitoring.svc.cluster.local:80
```

Because Grafana runs **inside the cluster**, not on the host.

Q86. How do you set up an alert in Grafana?

Answer:

1. Open a dashboard panel → **Edit** → **Alert** tab.
2. Create alert rule:
 - Query (metric)
 - Reduce (last/mean/min/max)
 - Threshold (e.g., > 9000)
 - Evaluation behavior (how long before firing)
 - Contact point (email, Slack, Jira)
3. Save.

Q87. What is a counter vs a gauge?

Answer:

- **Counter:** Monotonically increasing value (e.g., total requests). Never decreases.
- **Gauge:** Value that can go up or down (e.g., latency, CPU usage).

Q88. Why do we need a separate namespace for monitoring?

Answer: Isolation. If your app server crashes, monitoring keeps running. If monitoring crashes, your app keeps running. In production, monitoring runs on a dedicated server for the same reason.

Q89. How do you configure Prometheus to scrape a custom app in K8s?

Answer:

1. Get the app's Cluster IP and port: `kubectl get services .`
2. Edit Prometheus ConfigMap: `kubectl edit configmap prometheus-server -n monitoring .`
3. Add a `job_name` and `targets` under `scrape_configs` .
4. Restart Prometheus:
`kubectl rollout restart deployment prometheus-server -n monitoring .`

Q90. What is the difference between # HELP , # TYPE , and the metric line?

Answer:

text

```
# HELP <name> <description>    ← what the metric means
# TYPE <name> <type>           ← counter/gauge/histogram/summary
<name> <value>                 ← the actual data point
```

Q91. Why doesn't the metric value increase immediately when I hit the app?

Answer: Because Prometheus **scrapes every 15 seconds** by default. The new value appears after the next scrape.

Q92. What is the purpose of base64 decoding in Grafana's password?

Answer: Grafana stores the admin password as a **base64-encoded string** in a Kubernetes Secret. You must decode it to get the plain-text password:

bash

```
kubectl get secret grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64 --
decode
```

Q93. How do you build a dashboard in Grafana?

Answer:

1. **Dashboards** → **New** → **New Dashboard**.
2. **Add visualization** → select data source (Prometheus).
3. Enter PromQL query.
4. Choose visualization type (time series, gauge, table).
5. Save the dashboard.

Q94. What is the difference between monitoring locally vs on Kubernetes?

Answer:

- **Locally:** Prometheus/Grafana run as processes on your laptop.
- **On K8s:** They run as **pods** managed by Helm, in a **namespace**, with **cluster-internal DNS** for communication. Mirrors production.

Q95. What is the role of labels in PromQL?

Answer: Labels allow **filtering** and **aggregation**. Example:

```
text
```

```
total_api_requests_total{instance="10.107.61.186:5000"}
```

This filters the metric to a specific deployment. Useful when multiple apps expose the same metric.



TOPIC 16: Production & Deployment

Q96. What is the difference between Minikube and EKS/AKS/GKE?

Answer:

- **Minikube:** Single-node local cluster for learning.

- **EKS/AKS/GKE:** Managed Kubernetes services on AWS/Azure/GCP. The cloud provider manages the **control plane**; you manage worker nodes and workloads.

Q97. What is the advantage of managed Kubernetes?

Answer:

- Cloud provider handles the **control plane** (no need to hire a K8s admin).
- Auto-scaling, networking, monitoring built-in.
- Integrates with other cloud services (S3, RDS, etc.).

Q98. When would you use on-premises Kubernetes?

Answer: For **banks, government institutions**, or any organization with strict data compliance that can't use public cloud. Tools: `kubeadm` , `Rancher` , `k3s` .

Q99. How do you integrate Kubernetes into CI/CD?

Answer:

1. Build Docker image → push to ECR (same as before).
2. **Replace** "SSH + docker run" with `kubectl apply` against the K8s cluster.
3. Use GitHub Actions with `kubectl` CLI or a K8s action.

Q100. What are the production considerations for a K8s deployment?

Answer:

- **Resource limits:** Set CPU/memory requests and limits.
- **Replicas:** At least 2 for fault tolerance.
- **Health checks:** Liveness and readiness probes.
- **Secrets:** Use Kubernetes Secrets or AWS Secrets Manager.
- **Monitoring:** Prometheus + Grafana.
- **Ingress:** Set up an Ingress controller for external access.
- **Autoscaling:** HPA (Horizontal Pod Autoscaler).
- **Rolling updates:** Use Deployments for zero-downtime updates.

Final Tips for Interviews

1. **Be ready to explain the "why"** — not just the "what". Interviewers love candidates who understand trade-offs.
2. **Draw diagrams** — for K8s, monitoring, and CI/CD questions.
3. **Mention real tools** — Prometheus, Grafana, Helm, Docker, Kubernetes, AWS.
4. **Talk about production-grade practices** — environment variables, logging, exception handling, versioning.
5. **Know the MLOps lifecycle** — data ingestion → validation → transformation → training → evaluation → pusher → prediction → monitoring.
6. **Be honest about what you don't know** — say "I haven't worked with that, but based on what I know, I'd approach it like..."

End of Interview Questions & Answers.